応用情報技術者試験　テクノロジ系　#3

# コンピュータ構成要素

用語集まとめ ― 過去問33回・125問と規格団体の公開資料で確かめた図解・例題つき



点線の下線の用語・青い過去問番号・各ページ下の「詳しく:」を押すと、参照元のサイト（e-Words／応用情報技術者試験.com）で詳しい解説が開く

# 全体像 シラバスの5つの小分類で読む

**1 プロセッサ**
五大装置・命令の実行・アドレス指定・割込み・性能（CPI・MIPS）・パイプライン・並列処理

**2 メモリ**
DRAM・SRAM・フラッシュメモリ・記憶階層・キャッシュ（計算・書込み方式・対応付け）

**3 バス**
アドレス・データ・制御のバス・バスの調停・転送能力・PCI Express

**4 入出力デバイス**
USB・HDMI・ファイバチャネル・近距離無線・接続形態・DMA

**5 入出力装置**
ディスプレイ・プリンタ・磁気ディスクのアクセス時間・RAID

| 午前での出方 | |
|---|---|
| 問題数 | 毎回 **問8〜問12 のうち3〜5問**（R3春は問9〜問12）。10回で40問 |
| 同じ問題の再出題 | 5組（GPU、VLIW、実効アクセス時間、ライトスルーとライトバック、ユニファイドメモリ） |
| 問題の型 | 計算問題が約4割（CPI・MIPS・パイプライン・キャッシュ・ディスク）。残りは「説明を選ぶ」で、誤りの選択肢は同じ仲間の技術 |

**この資料の読み方：**1枚に1つのメッセージ。見出しの一文と、図の中の数字を自分で計算し直せれば解けるようにしてある。



詳しく：応用情報.com 用語集 3-2 メモリ ／ 応用情報.com 用語集 3-1 プロセッサ

# 過去10回・40問の内訳 令和3年度春期〜令和7年度秋期

| テーマ | 数 | よく出る論点 |
|---|---|---|
| キャッシュメモリ | 8 | ライトスルーとライトバック（3回）、実効アクセス時間（2回）、ヒット率の逆算、2段キャッシュ、フルアソシエイティブ |
| パイプライン・高速化 | 6 | VLIW（2回）、必要なサイクル数、5段の順序、投機実行、ヘテロジニアスマルチコア |
| 性能の計算 | 5 | CPI（2回）、命令ミックスのMIPS、FLOPS、タイマのカウント |
| 入出力装置 | 4 | ディスクの平均アクセス時間、RAID5の容量、有機EL、電子ペーパー |
| フラッシュ・主記憶 | 6 | ウェアレベリング（2回）、ユニファイドメモリ（2回）、インタリーブ、展開時間 |
| 入出力インタフェース | 3 | USB Type-C、ファイバチャネル、DMA |
| 並列・GPU・量子 | 6 | アムダールの法則（2回）、GPU（2回）、量子ゲート方式、量子超越性 |
| 演算と命令 | 2 | 符号付き加算のオーバーフロー、重なったブロック転送 |

**一番の得点源：**キャッシュメモリ。10回で8問。「実効アクセス時間の式」と「ライトスルーとライトバックの違い」の2つで、ほぼ全部解ける。

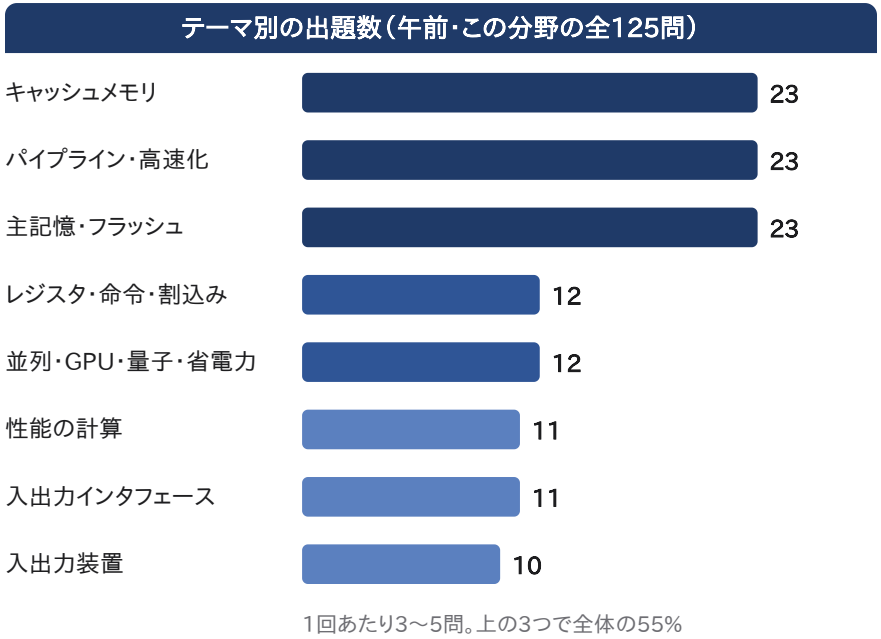
**計算問題：**CPI・MIPS・パイプライン・キャッシュ・ディスクの5つの式を覚えれば、計算は全部この中に入る。

**落とし穴：**説明を選ぶ問題は、誤りの選択肢が「仲間の別技術の正しい説明」になっている（DMAの問題にDSP・MMU・タイマの説明が並ぶなど）。



詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 3-2 メモリ

# 33回・125問で見ると、キャッシュ・高速化・主記憶が3本柱 H21春〜R7秋




1回あたり3〜5問。上の3つで全体の55%

**何度も出た論点（同じ論点の回数）**

| 論点 | 回数 | この資料 |
|---|---|---|
| 実効アクセス時間・ヒット率 | 9 | キャッシュの計算 |
| ライトスルーとライトバック | 9 | ダーティビットの例題 |
| CPI・MIPS・FLOPS の計算 | 9 | 命令ミックスの例題 |
| メモリインタリーブ | 8 | バンクのタイミング図 |
| パイプラインのサイクル数・図 | 8 | k+(n−1) の式 |
| SIMD・MIMD・GPU | 7 | 4つを一度に足す図 |
| スーパースカラ／VLIW | 5／5 | 判断するのは誰か |
| スタックポインタ・PC | 5 | レジスタで追う例題 |
| ハミング符号・DMA・USB | 4ずつ | 手で直す・流れ図・世代表 |

**使い方：**回数の多い論点ほど、そっくりの問題がまた出る。平成の問題も令和の問題も、選択肢の顔ぶれはほとんど変わらない。

集計：IPA 応用情報技術者試験 午前 問題冊子（IPA 過去問題）33回分の問7〜問13 を読み、分野は応用情報技術者試験.comの分類で確かめた。論点の分け方と数はClaudeによる

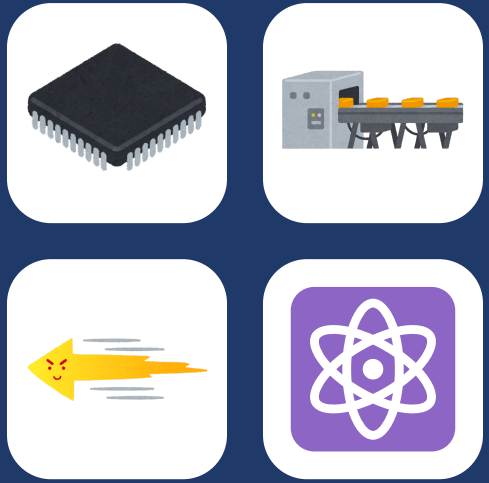


詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 3-2 メモリ

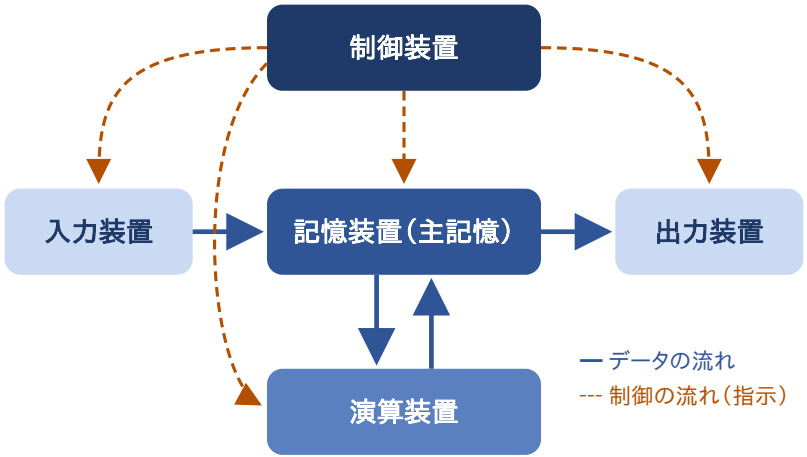
第1章

# プロセッサ

五大装置 ／ 命令の実行とレジスタ ／ アドレス指定 ／ 割込み ／ 性能の計算 ／ パイプライン ／ 並列処理 ／ 量子コンピュータ



# コンピュータは5つの装置。データは記憶装置を通り、指示は制御装置から出る




| 装置 | 役目 |
|---|---|
| 制御装置 | 命令を取り出して解読し、全部の装置に指示を出す |
| 演算装置 | 計算する（加算器・レジスタなど） |
| 記憶装置 | プログラムとデータを置く。プロセッサが直接読むのは主記憶 |
| 入力装置 | キーボード・マウス・スキャナなど |
| 出力装置 | ディスプレイ・プリンタなど |

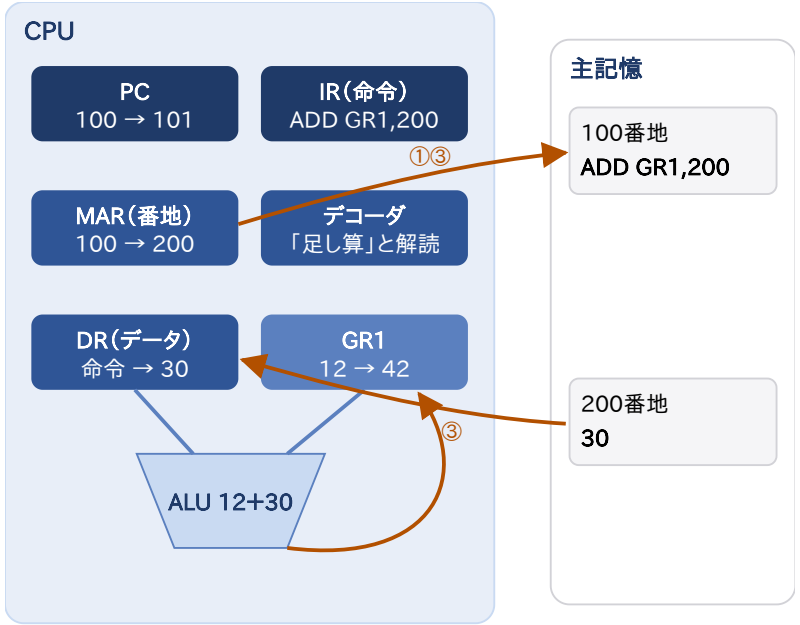
**制御装置＋演算装置＝プロセッサ（CPU）。**入力したデータは、いったん主記憶に入ってから計算される（入力装置から演算装置へ直接は行かない）。

プログラムもデータと同じ主記憶に置いて順に読み出して実行する方式を**プログラム内蔵方式**（ノイマン型）と呼ぶ。



詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 3-5 入出力装置

# 命令1つの一生をレジスタで追う:「GR1 に 200番地の値を足す」




1. **命令の取出し(フェッチ)**:PC の100を MAR に移して主記憶を読む。届いた命令は DR を通って IR へ。PC は次の101に進める
2. **解読(デコード)**:デコーダが IR の命令部を読み「足し算・相手は200番地」と分かる
3. **データの取出し**:アドレス部の200(有効アドレス)を MAR に入れて読む。DR に 30 が届く
4. **実行**:ALU が GR1 の12と DR の30を足し、42を GR1 に書く。結果がゼロか・負か・あふれたかはフラグ(PSW)に残る

> **過去問の選択肢はこの役割分担から作られる:**「次の命令の番地」=PC、「読み出した命令」=IR、「読み出したデータ」=DR、「演算結果の状態」=フラグ(R1 秋 問9・H29春 問8)。

出典:IPA シラバス Ver.7.2 中分類3(5)プロセッサの構造と方式・(6)②命令の実行手順。数値はClaudeが作った例

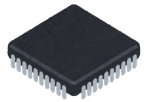
# 🧠 プロセッサは「何を一斉に計算するのが得意か」で使い分ける

| 種類 | 得意なこと | しくみ |
|---|---|---|
| CPU | 分岐の多い複雑な処理・全体の制御 | 少数の高性能なコア |
| GPU | 画像処理・**行列演算**（ディープラーニング） | 単純な演算器を数千個並べて一斉に |
| GPGPU | GPUの並列計算を画像以外に使う「使い方」 | — |
| DSP | 音声・画像の信号処理 | 積和演算（a×b+c）を1命令で |
| FPU | 浮動小数点数の演算 | CPUを助けるコプロセッサ |
| AIチップ・TPU | 機械学習の学習・推論 | 精度を落とした行列計算を大量に |

**R3春 問10＝R4秋 問8：ディープラーニングにGPUを使う利点**

| | | |
|---|---|---|
| ア | 各コアが独立に異なるプログラムを実行 | MIMD（マルチコアCPU） |
| **イ** | **行列演算ユニットで行列演算を高速に** | **正解** |
| ウ | 浮動小数点演算ユニットをコプロセッサに | FPU |
| エ | 分岐予測でパイプラインの効率を上げる | CPUの高速化技術 |

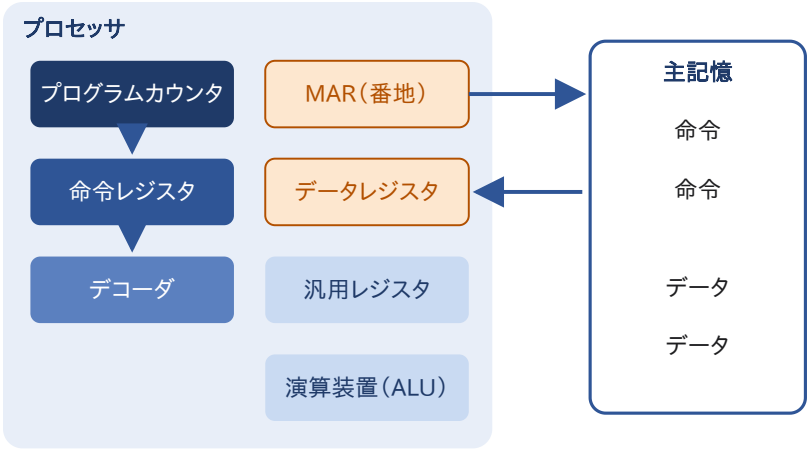
誤りの選択肢は、どれも**ほかのプロセッサの正しい説明**。「GPU＝同じ計算を一斉に（SIMD）」と覚えておけば、ア（別々のプログラム）とエ（分岐）が外れる。





詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 1-3 情報に関する理論

# 命令は「取り出す→解読→実行」の繰返し。番地とデータは別のレジスタ




| 順 | 段階（RISCの5段） | 中身 |
|---|---|---|
| 1 | 命令フェッチ | プログラムカウンタの番地から命令を読み、命令レジスタへ |
| 2 | デコード・レジスタ読出し | デコーダが命令部を解読。使うレジスタの値を読む |
| 3 | 実行・アドレス生成 | ALUで計算。または有効アドレスを計算 |
| 4 | メモリアクセス | 番地はMAR、データはデータレジスタで読み書き |
| 5 | 書込み | 結果をレジスタに書き込む |

R6秋 問8：RISCの5段の順序は ④フェッチ→③デコード→②実行→⑤メモリアクセス→①書込み（**ウ**）。番地が決まる前にメモリは読めない、で並びが決まる。

# 📦 スタックは「最後に積んだものから出す」。スタックポインタは一番上の番地

**メイン → 関数A → 関数B と呼び、Bから戻るまで**




呼ぶたびに戻り先を積み（プッシュ）、戻るときに一番上から取り出す（ポップ）。だから入れ子の呼出しも、再帰呼出しも、正しい順番で戻れる。割込みのときも、元のPCやレジスタをここに退避する。

**レジスタの役割（選択肢に並ぶ顔ぶれ）**

| レジスタ | 中身 |
|---|---|
| **スタックポインタ** | スタックの一番上の番地 |
| プログラムカウンタ | 次に取り出す命令の番地 |
| 命令レジスタ | 取り出した命令そのもの |
| PSW・フラグ | 演算結果の状態、割込みの許可 |
| MAR／DR | 読み書きする番地／データ |

**3回同じ問題：**H24秋 問10・H28春 問8・R2秋 問8「スタックポインタが示すもの」。誤りの3つは PC・IR・PSW の中身。

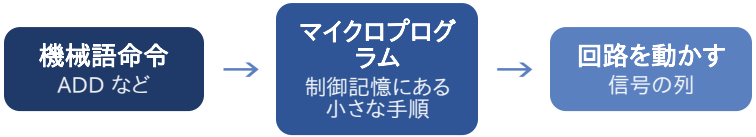
出典：IPA シラバス Ver.7.2 中分類3（5）用語例「スタックポインタ」／IPA 平成24年度秋期・平成28年度春期・令和2年度10月 午前 問題。番地はClaudeが作った例

# RISCとCISC：命令をそろえて速く回すか、1命令に多くを詰めるか

| | RISC | CISC |
|---|---|---|
| 命令の長さ | **固定長**（例：32ビット） | **可変長**（短い命令も長い命令も） |
| 命令の数と中身 | 少なく単純。メモリを触るのはロードとストアだけ | 多く複雑。1命令でメモリの値を直接計算できる |
| 制御のしかた | 回路で直接（ワイヤードロジック） | マイクロプログラム制御が多い |
| パイプライン | 段をそろえやすく向いている | 中で単純な命令に分けてから流す |
| 実物の例 | Arm（スマートフォン、Apple M1）、RISC-V | インテル・AMD の x86 |

RISC-V は「誰でも使用料なしで使える、オープンな命令セットの規格」（RISC-V International）。命令セット（ISA）は、ソフトとハードの間の約束ごとの一覧のこと。

**マイクロプログラム制御**

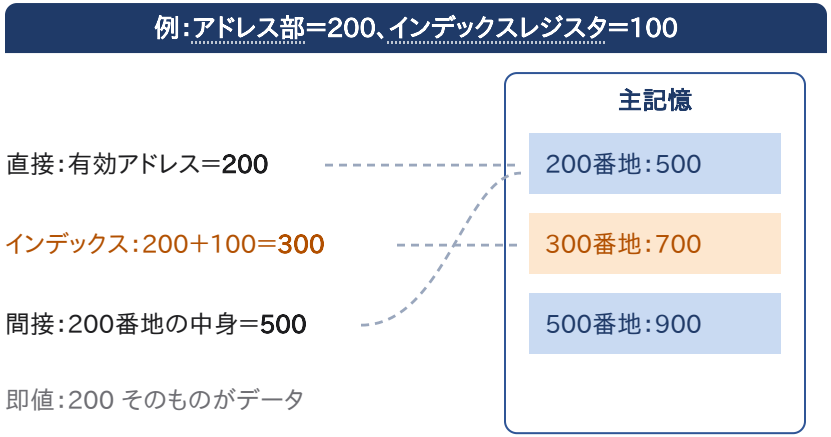



| 種類 | 特徴 |
|---|---|
| 水平型 | 1語が長く、多くの回路を同時に直接動かす。速い |
| 垂直型 | 1語が短く、解読してから動かす。メモリが少なくて済む |

マイクロプログラムを書き換えると、別のコンピュータの命令をまねて動かせる（エミュレーション）。

出典：IPA シラバス Ver.7.2 中分類3（4）③命令セット・（7）マイクロプログラム制御／RISC-V International「About」／Apple Newsroom「Apple unleashes M1」(2020)／Wikipedia「Reduced instruction set computer」／「Microcode」

# アドレス指定は「アドレス部の値に何を足すか、何回たどるか」の違い




| 方式 | 有効アドレス | 使いどころ |
|---|---|---|
| 即値 | （アドレス部がデータ） | 定数 |
| 直接 | アドレス部 | 決まった変数 |
| 間接 | アドレス部の番地の中身 | ポインタ |
| インデックス | アドレス部＋指標レジスタ | 配列の i 番目 |
| ベース | ベースレジスタ＋アドレス部 | プログラムの再配置 |
| 相対 | プログラムカウンタ＋アドレス部 | 近くへの分岐 |

> 「有効アドレス」は**番地**。その番地に入っている**データ**（例では700）と取り違える誤りの選択肢が定番。

# アドレス指定は、1枚のメモリ表で6つとも計算できる

**例題** 命令「LD GR0, 100」（アドレス部＝100）。インデックスレジスタ＝3、ベースレジスタ＝500、PC（次の命令の番地）＝40。

| 番地 | 中身 |
|---|---|
| 100 | 250 |
| 103 | 77 |
| 140 | 55 |
| 250 | 999 |
| 600 | 12 |

| 方式 | 有効アドレスの出し方 | 有効アドレス | GR0 に入る値 |
|---|---|---|---|
| 即値 | アドレス部がデータそのもの | （なし） | **100** |
| 直接（絶対） | アドレス部のまま | 100 | **250** |
| 間接 | 100番地の中身を番地として使う | 250 | **999** |
| インデックス | 100 ＋ インデックス 3 | 103 | **77** |
| ベース | ベース 500 ＋ 100 | 600 | **12** |
| 相対 | PC 40 ＋ 100 | 140 | **55** |

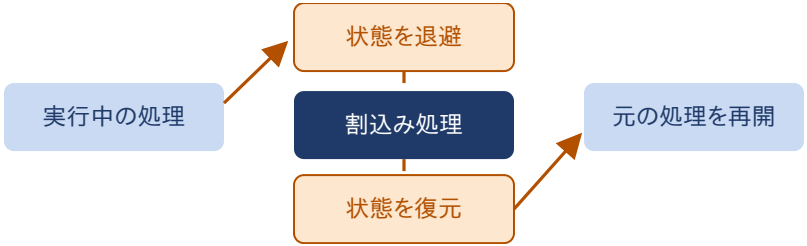
**2段で考える：**まず「有効アドレス（データのある番地）」を出し、次に「その番地の中身」を読む。問題が番地を聞いているのか、データを聞いているのかで答えが変わる。

**過去問：**H26春 問10（絶対アドレス＝アドレス部をそのまま使う）、H28秋 問9（間接＝「アドレスが入っている場所のアドレス」）。どちらも誤りの選択肢はこの表のほかの行。

出典：IPA シラバス Ver.7.2 中分類3（6）②アドレス修飾の用語例／IPA 平成26年度春期 午前 問10・平成28年度秋期 午前 問9。数値はClaudeが作った例題

# 🔔 割込みは「原因がプログラムの中か外か」で内部と外部に分かれる

| 分類 | 種類 | 原因 |
|---|---|---|
| **内部**（同期） | プログラム割込み | 0で割った、オーバーフロー、不正な命令・番地、ページフォールト |
| | SVC割込み | OSの機能を呼び出す命令（システムコール） |
| **外部**（非同期） | 入出力割込み | 入出力装置の転送の完了・異常 |
| | タイマ割込み | 設定した時間の経過（タスクの切替え） |
| | マシンチェック割込み | 電源の異常・メモリの故障 |
| | コンソール割込み | オペレータの操作 |




**マスク可能割込み**は設定で一時的に止められる。電源異常のように必ず受け付けるものは**マスク不能割込み（NMI）**。

**多重割込み：**割込み処理の最中に、優先度の高い割込みだけを受け付ける。低いものは待たせる。

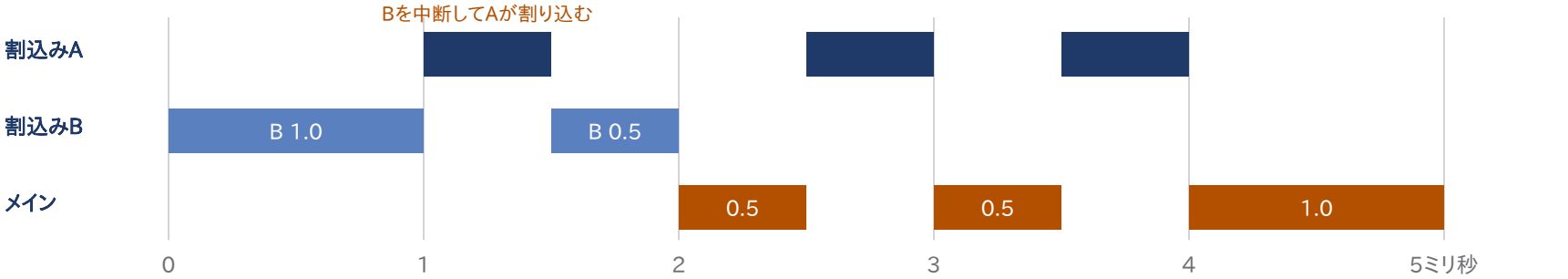
「内部＝プログラムの中の原因」。入出力の完了やタイマは、プログラムが動かしたきっかけでも**外部**割込み。



詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 5-1 オペレーティングシステム

# 🔔 多重割込みはタイムラインに描けば解ける

**H26春 問9** 割込みA（0.5ミリ秒・優先度 高）は 1.0・2.5・3.5 ミリ秒に、割込みB（1.5ミリ秒・優先度 低）は 0・5.0 ミリ秒に発生。0〜5ミリ秒でメイン処理が使えるCPU時間は？




**答え：**メインは 0.5+0.5+1.0=**2ミリ秒（ア）**。高い優先度のAは、実行中のBを中断して先に動く。5.0ミリ秒のBは範囲の外。

**H26春 問8（割合の型）：**100MIPS のCPUで、1ミリ秒ごとに1万命令の割込み処理。1ミリ秒に10万命令なので割込みが10%。残りは **90MIPS 相当**。

出典：IPA 平成26年度春期 応用情報技術者試験 午前 問8・問9（表と図をClaudeが図に描き直した）

# ⏱️ 性能の計算は「1命令の平均時間」を出してから逆数にする

**1命令の平均時間 ＝ CPI × サイクルタイム**
サイクルタイム＝1÷クロック周波数

**MIPS ＝ 1 ÷ 1命令の平均時間 ÷ $10^6$**

| 過去問 | 計算 | 答え |
|---|---|---|
| R5春 問8 | 1.25GHz で毎秒10億命令 → 12.5億÷10億 | CPI 1.25 |
| R7春 問8 | A：4.0×1ns＝4ns、B：0.5×4ns＝2ns | Aは2倍 |
| R7秋 問11 | 20GFLOPS × 32,000コア | 640TFLOPS |
| R7秋 問10 | 1MHzで10,000カウント＝10ms、1回転＝2パルス＝20ms | 毎秒50回転 |

**R3春 問9：命令ミックスで MIPS を求める**

| 命令 | 実行時間 | 出現頻度 | 時間×頻度 |
|---|---|---|---|
| 整数演算 | 10ns | 50% | 5ns |
| 移動 | 40ns | 30% | 12ns |
| 分岐 | 40ns | 20% | 8ns |
| **1命令の平均** | | | **25ns** |

**1 ÷ 25ns ＝ $4\times10^7$ 命令/秒 ＝ 40MIPS（ウ）**

誤りの選択肢は計算の途中の数（25）や、重みを付けずに足した数（90、その逆数の11）。**最後に逆数にしたか**を確かめる。

# CPIは命令ミックスの加重平均。3つの手順で MIPS まで行く

**H22秋 問9** 700MHz のCPU。命令の種類ごとのクロック数と出現率は下の表。約何MIPSか。

| 命令 | クロック数 | 出現率 | クロック×率 |
|---|---|---|---|
| レジスタ間演算 | 4 | 30% | 1.2 |
| メモリ・レジスタ間演算 | 8 | 60% | 4.8 |
| 無条件分岐 | 10 | 10% | 1.0 |
| **平均（CPI）** | | | **7** |

1. **CPIを出す**：クロック数×出現率を足す → **7クロック／命令**
2. **1秒のクロック数**：700MHz → **7億クロック／秒**
3. **割る**：7億 ÷ 7 ＝ 1億命令／秒 ＝ **100MIPS（エ）**

**MIPS ＝ クロック周波数（MHz）÷ CPI**

時間で考えても同じ。1クロック＝1÷700M≒1.43ナノ秒、1命令＝7×1.43≒10ナノ秒、1÷10ナノ秒＝1億命令／秒。

**誤りの選択肢**：70 は700を一番遅い分岐の10で割った値（重み付けなし）。10 は桁の取り違え。

出典：IPA 平成22年度秋期 応用情報技術者試験 午前 問9（表をClaudeが書き直した）／Wikipedia「Cycles per instruction」／「Instructions per second」

# ⏱️ 処理時間＝命令数×CPI×サイクルタイム。比べる問題は3回出た

**CPU時間 ＝ 命令数 × CPI × サイクルタイム（＝1÷クロック周波数）**

**例題** 同じ命令セットのコンピュータ A と B。A：クロック周期 1ナノ秒・CPI 4.0、B：クロック周期 4ナノ秒・CPI 0.5。A の処理時間は B の何倍か。

1. 同じ命令セット・同じプログラムなので**命令数は同じ**。比べるのは「CPI×周期」だけ
2. A：4.0×1＝4ナノ秒／命令、B：0.5×4＝2ナノ秒／命令
3. 4÷2＝**2倍**（A の方が遅い）

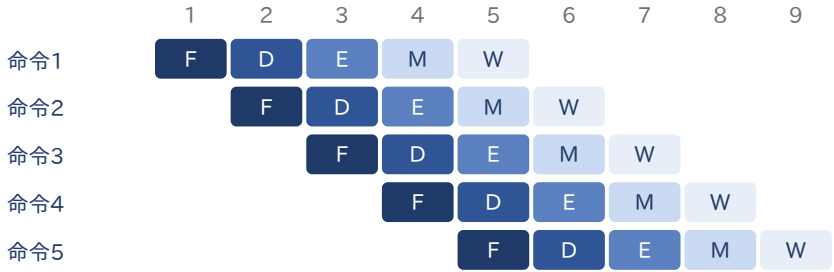
| 出た回 | 型 |
|---|---|
| H22春 問10・H24春 問12・R7春 問8 | この例題と同じ型（3回） |
| R5春 問8 | 1.25GHz で毎秒10億命令 → CPI＝1.25 |

| 単位の早見 |
|---|
| k＝$10^3$　M＝$10^6$　G＝$10^9$　T＝$10^{12}$ |
| m＝$10^{-3}$　μ＝$10^{-6}$　n＝$10^{-9}$　p＝$10^{-12}$ |
| 1GHz ⇔ 1ナノ秒、1MHz ⇔ 1マイクロ秒 |

**落とし穴：**「クロックが速い方が速い」とは限らない。CPI が大きいと逆転する。誤りの選択肢 1/32 や 8 は、片方だけで比べた値。

出典：IPA 平成22年度春期 午前 問10・平成24年度春期 午前 問12・令和7年度春期 午前 問8・令和5年度春期 午前 問8。例題の数値は令和7年度春期 問8の形に合わせてClaudeが書いた

# 🚀 パイプラインは段をずらして重ねる。k段で n命令なら k+(n−1) サイクル




F=フェッチ　D=デコード　E=実行　M=メモリ　W=書込み

**1つ目が5サイクルで終わったら、あとは1サイクルに1つずつ終わる**

5命令なら 5+(5−1)=9 サイクル（重ねないと 5×5=25）

**R5春 問9：5ステージで20命令**

$$5 + (20 - 1) = 24\text{サイクル（ウ）}$$

誤りの選択肢：20（最初の命令がパイプラインを満たす時間を入れていない）、21、25（5+20で1つ目を二重に数えた）。

1命令にかかる時間は短くならない。速くなるのは「まとめて処理する速さ（スループット）」。



# パイプラインが止まる原因は3つ。対策は「予測する・順番を入れ替える」

| ハザード | 起きること | 主な対策 |
|---|---|---|
| データハザード | 前の命令の結果がまだできていない | 結果を直接渡す（フォワーディング） |
| 構造ハザード | 同じ回路・メモリを取り合う | 命令用とデータ用のキャッシュを分ける |
| 制御ハザード | 分岐先が決まらず、読んだ命令が無駄になる | 分岐予測・投機実行・遅延分岐 |

3つまとめて**パイプラインハザード**。起きると、パイプラインに空き（バブル）ができて効果が下がる。

| R5秋 問9：分岐が決まる前に、予測した先を実行する技術 | | |
|---|---|---|
| ア | アウトオブオーダー実行 | 関係のない後ろの命令を先に実行 |
| イ | 遅延分岐 | 分岐命令の直後の命令を必ず実行して空きを埋める |
| **ウ** | **投機実行** | **予測した先を実行し、外れたら取り消す（正解）** |
| エ | レジスタリネーミング | 名前だけ同じレジスタを別の実体に割り当てる |

**分岐予測**は「予想する」だけ。予想した先を**実行まで進める**のが投機実行。選択肢に両方並んだら、この違いで選ぶ。

# ハザードを図で見る:データは「待つか横流し」、分岐は「捨ててやり直す」

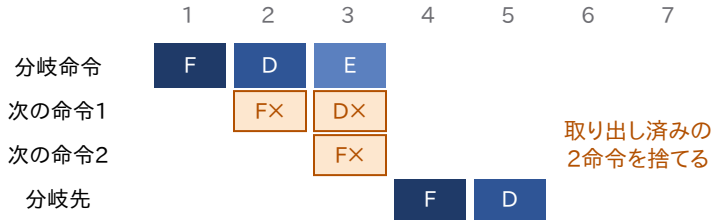
## データハザード:SUB が ADD の結果 R1 を使う




R1 が書かれる(W)まで解読を待つ → 3サイクルの空き(バブル)

**対策:**フォワーディング(E段の結果を、レジスタに書く前に次の命令へ横流しする)、関係のない命令を間に入れる(並べ替え)。

## 制御ハザード:分岐の結果が E段で分かる



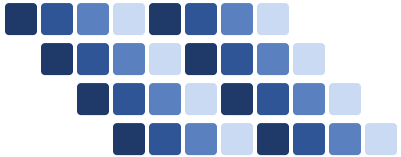

**対策:**分岐予測+投機実行(先に予測した方を進めておく)、分岐を減らすプログラム(H27秋 問8)。

**例題** 命令の20%が分岐、そのうち予測が外れるのが10%、外れると2サイクル失う。理想の CPI が1のとき、実際の CPI は? → 1 + 0.2 × 0.1 × 2 = **1.04**(4%遅くなる)。予測が当たらないと 1 + 0.2 × 2 = 1.4。

出典:IPA 平成25年度春期 午前 問10(制御ハザード=分岐命令)・平成26年度秋期 午前 問7・平成27年度秋期 午前 問8／Wikipedia「Hazard (computer architecture)」／「Branch predictor」。例題の数値はClaudeが作った

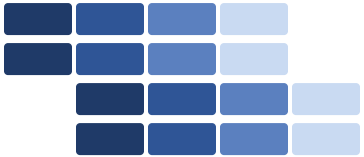
# もっと速くする方法は「段を細かく」「本数を増やす」「まとめ役をコンパイラに」

## スーパーパイプライン



1段を細かく分けて**段数を増やす**。1段が短くなるのでクロックを上げられる。

## スーパースカラ



パイプラインを**複数本**もち、同時に流せる命令を**ハードウェアが実行時に**見つける。

## VLIW




1つのとても長い命令に
同時にできる動作を詰める

同時にできる動作を**コンパイルの段階で**1つの長い命令にまとめる。

R4春 問8=R7秋 問8:「同時に実行可能な複数の動作を、コンパイルの段階でまとめて一つの複合命令とする」→ **VLIW（エ）**。誤りはCISC・MIMD・RISC。

見分けどころは**だれが並べるか**。実行時にハードウェア＝スーパースカラ、コンパイル時にコンパイラ＝VLIW。

# 投機実行・アウトオブオーダー・SMT は「空いている時間を何で埋めるか」

| アウトオブオーダー実行の例 | | |
|---|---|---|
| # | 命令 | 状況 |
| 1 | LD R1, [a] | キャッシュミスで約80ナノ秒待つ |
| 2 | ADD R2, R1, 1 | 1の結果が要る → 待つ |
| 3 | MUL R3, R4, R5 | 1・2と無関係 |
| 4 | SUB R6, R7, R8 | 1・2と無関係 |

**順番どおり**
3・4も1を待つ

→

**順番を入れ替え**
待つ間に3・4を先に実行

結果をレジスタに書き込む順番だけは、プログラムの順に戻してから確定する。だから外から見ると順番どおりに動いたのと同じになる。

| 技術 | 空きを埋めるもの | 外れたら |
|---|---|---|
| 分岐予測 | 分岐先を過去の結果から当てる | — |
| **投機実行** | 予測した分岐先の命令を先に実行 | 結果を捨てる |
| アウトオブオーダー | 後ろの無関係な命令 | （予測しないので外れない） |
| SMT（ハイパースレッディング） | **別のスレッド**の命令 | — |

**SMT：**インテルは「1つの物理コアを2つの論理コアのように動かす技術」と説明している。演算器の空きに別スレッドの命令を入れるので、コアが2倍になるわけではない。

**過去問：**H27春 問10（投機実行）の誤りの選択肢は、ア＝アウトオブオーダー、イ＝SMT、ウ＝マルチコア。R5秋 問9も同じ顔ぶれ。

出典：IPA 平成27年度春期 午前 問10・令和5年度秋期 午前 問9／インテル「ハイパースレッディングとは？」／Wikipedia「Out-of-order execution」・「Speculative execution」。待ち時間約80ナノ秒はメモリ編の記憶階層の表（DDR5の実測例 82.5ナノ秒）から

# コアの組合せは「同じ種類か、違う種類か」「役割を決めるか、対等か」

| 用語 | 意味 |
|---|---|
| ホモジニアスマルチコア | **同じ**種類のコアを複数（PC・サーバ） |
| ヘテロジニアスマルチコア | **違う**種類のコアを組み合わせる（スマートフォン） |
| big.LITTLE | 高性能コア＋省電力コアの組合せ（ヘテロの代表） |
| マルチスレッディング | 1つのコアで複数のスレッドを並べて実行 |
| SMP（対称型） | どのプロセッサも対等に、OSもアプリも分担 |
| AMP（非対称型） | OS用・アプリ用など役割を決めて分担 |
| 密結合／疎結合 | 主記憶を共有する／各自の主記憶とOSで通信 |

**R5秋 問8：性能と低消費電力の両立のため、違う種類のコアを載せる**

| | | |
|---|---|---|
| ア | スーパースカラ | パイプラインを複数本（コアの種類の話ではない） |
| イ | ソフトコア | FPGAの上に作るプロセッサ |
| **ウ** | **ヘテロジニアスマルチコア** | **正解**（hetero＝異なる） |
| エ | ホモジニアスマルチコア | homo＝同じ。「複数の種類」に合わない |

> 重い処理は高性能コア、待受けや軽い処理は省電力コア。スマートフォンの電池が長くもつ理由の一つ。



# 省電力は「何を止めるか」で見分ける。電圧を下げるのが一番効く

| 技術 | 止める・変えるもの | 効き方 |
|---|---|---|
| DVFS | 仕事が少ないとき**電圧と周波数**を下げる | 電力は電圧の2乗に効く |
| クロックゲーティング | 使わない回路への**クロック**を止める | 切替えの電力が0。漏れ電流は残る |
| **パワーゲーティング** | 使わない回路の**電源**を切る | 漏れ電流まで0。戻すのに時間 |
| 電力枠の融通 | 止めたコアの余裕で動くコアのクロックを上げる | 速くするための技術 |
| big.LITTLE | 軽い仕事は省電力コアに回す | コアを使い分ける |

**Arm の説明：**big.LITTLE は最大3種類のコアを使い分けるヘテロジニアス（異種）構成。LITTLE は電力効率最優先、big は持続的な性能のためのコア。

**動的な電力 ∝ 容量 × 電圧² × 周波数**

**例題** 電圧を 1.0V→0.8V、周波数を 2.0GHz→1.6GHz に下げた。電力は何倍か。

1. 電圧の比 0.8 を2乗 → 0.64
2. 周波数の比 → 0.8
3. 0.64 × 0.8 = **約0.51倍**（速さは0.8倍なのに電力は半分）

**過去問：**H29春 問9（パワーゲーティング）の誤りは、ア＝DVFS、イ＝クロックゲーティング、エ＝電力枠の融通。R5秋 問8はヘテロジニアスマルチコア。

出典：IPA 平成29年度春期 午前 問9・令和5年度秋期 午前 問8／Arm「big.LITTLE」／Wikipedia「Processor power dissipation」（動的電力は周波数と電圧の2乗に比例）・「Power gating」・「Dynamic frequency scaling」

# フリンの分類は「命令の流れ」と「データの流れ」が1つか複数か

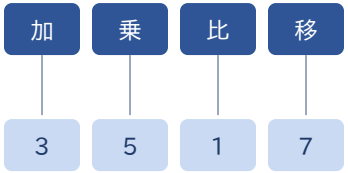
| | データ 1つ | データ 複数 |
|---|---|---|
| 命令 1つ | SISD<br>従来の単一プロセッサ | SIMD<br>GPU・ベクトル処理・アレイコンピュータ |
| 命令 複数 | MISD<br>実用例はほとんどない | MIMD<br>マルチプロセッサ・マルチコア |

S=Single（1つ）、M=Multiple（複数）、I=Instruction（命令）、D=Data（データ）。前半が命令、後半がデータ。

SIMD：1つの命令を全員に

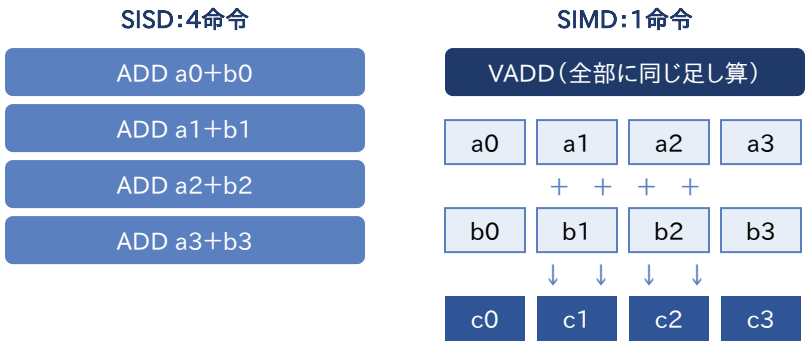



MIMD：別々の命令で別々に




GPUが行列演算に強いのは、SIMDで同じ計算を一斉に行うから

# 🚀 SIMDは1命令で4つの足し算。GPUとTPUはこれを数千並べたもの




1,024個の足し算：SISD は1,024命令、8個ずつの SIMD なら128命令

| 実物 | どんなSIMD／並列か |
|---|---|
| CPU のベクトル命令 | インテル・AMD の AVX、Arm の NEON など。1命令で複数の値を処理 |
| GPU | 単純な演算器を数千個並べ、同じ計算を一斉に行う。行列の計算が速い（R3春 問10） |
| TPU | Google の機械学習専用の ASIC。行列の掛け算を行う MXU（シストリックアレイ）が中心 |
| MIMD | コアごとに**別の命令**。マルチコア、クラスタ（H26秋 問8） |

**Google の説明：**行列の計算が中心のモデルは TPU で速いが、足し算・並べ替えなど行列以外の処理が多いと MXU を使い切れない。**得意な計算を一斉にやる**のが SIMD 系の共通点。

SIMD は H21春・H25春・H27秋・H28春の4回。選択肢はフリンの分類の4つ。

出典：IPA 平成21年度春期 問9・平成25年度春期 問11・平成27年度秋期 問9・平成28年度春期 問9・平成26年度秋期 問8・令和3年度春期 問10（午前）／Google Cloud「Introduction to Cloud TPU」／Wikipedia「Single instruction, multiple data」

# ⏳ アムダールの法則：並列にできない部分が、速くなる上限を決める

**速度向上比 ＝ 1 ÷ { (1−r) ＋ r ／ n }**

r＝並列化できる割合　n＝プロセッサ数

| n | r=0.9 | r=0.3 | 比 |
|---|---|---|---|
| 1 | 1.00 | 1.00 | 1.0倍 |
| 3 | 2.50 | 1.25 | 2.0倍 |
| 6 | **4.00** | **1.33** | **3.0倍** |
| ∞ | 10（=1÷0.1） | 1.43（=1÷0.7） | — |

R7春 問11：r=0.9 が r=0.3 の3倍になるのは **6台（エ）**。0.7+0.3/n=3(0.1+0.9/n) を解いて n=6。

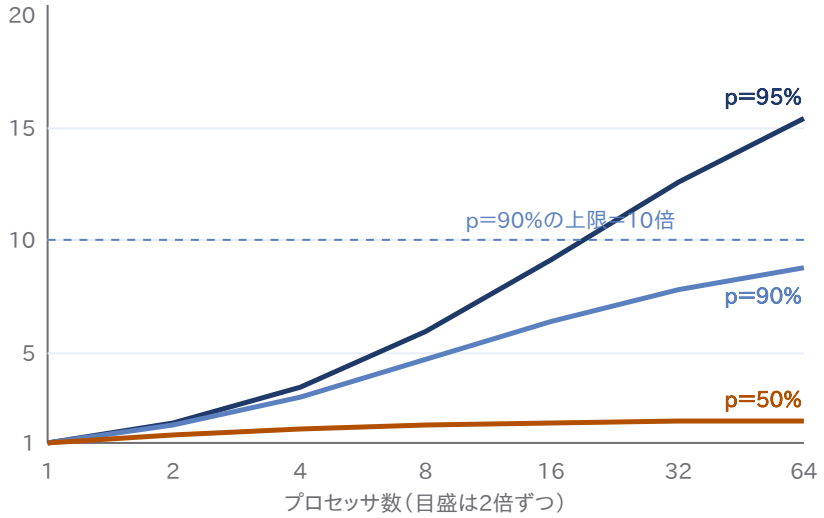
**R4春 問12：アムダールの法則に基づく記述**

| | | | |
|---|---|---|---|
| ア | 全部並列化できる→ある水準に近づく | × | 全部できるなら n に比例して増え続ける |
| イ | 並列化できない処理がある→n に比例 | × | 比例するのは全部できるときだけ |
| **ウ** | **並列化できない処理がある→ある水準に近づく** | **正解** | 上限は 1÷(1−r) |
| エ | 並列化できる割合が増える→反比例で減る | × | 割合が増えれば速くなる |

並列にできない部分の例：主記憶の取り合い、データベースのロック待ち（シラバスの「並列処理の隘路」）。



# アムダールの法則を数字で:90%並列にできても、上限は10倍




**速度向上比 ＝ 1 ÷ {(1−p) ＋ p÷n}**

p＝並列にできる割合、n＝プロセッサ数

| n | p=0.5 | p=0.9 | p=0.95 |
|---|---|---|---|
| 2 | 1.33 | 1.82 | 1.90 |
| 8 | 1.78 | 4.71 | 5.93 |
| 64 | 1.97 | 8.77 | 15.4 |
| ∞ | 2 | 10 | 20 |

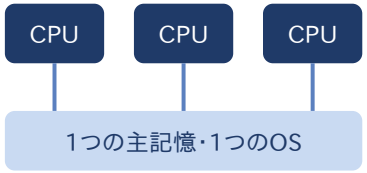
**手順** p=0.9、n=8：並列部分 0.9÷8=0.1125、逐次部分 0.1。足して 0.2125、逆数で **約4.7倍**。nを無限にしても 1÷0.1=10倍で止まる。

**過去問:**R4春 問12（グラフの読み取り）・R7春 問11（式に代入）。

出典:IPA 令和4年度春期 午前 問12・令和7年度春期 午前 問11／Wikipedia「Amdahl's law」（G. Amdahl, AFIPS 1967）。表とグラフの値はClaudeが式から計算した

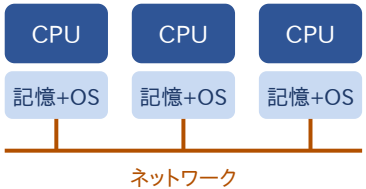
# 🔗 マルチプロセッサのつなぎ方は「メモリを共有するか」で2つに分かれる

**密結合（共有メモリ）**




例：SMP、今のマルチコアCPU

**疎結合（それぞれ別）**

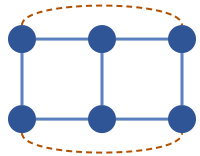



例：クラスタ、スーパーコンピュータ

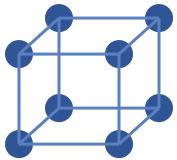
プロセッサが増えると、共有メモリは取り合い（隘路）になりやすい

| 用語 | 意味 |
|---|---|
| SMP | 対等なCPUが1つの主記憶を共有（密結合） |
| NUMA | CPUごとに近いメモリ・遠いメモリがあり、遠いと遅い |
| タンデム | 処理を段に分けて、前のCPUの結果を次のCPUが受け継ぐ |

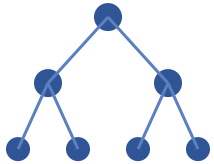
**つなぎ方（トポロジ）**



トーラス



ハイパーキューブ



ハイパーツリー

**実物：2025年11月 TOP500 の1位**

| | |
|---|---|
| El Capitan（米ローレンス・リバモア研究所） | **1.809 エクサFLOPS** |
| コア数 | 1,134万（AMD EPYC+ MI300A） |
| 電力あたりの性能 | 60.9 ギガFLOPS／W |

> エクサ＝$10^{18}$。4位の JUPITER（独）が米国以外で初めて1エクサFLOPSに届いた。

出典：IPA シラバス Ver.7.2 中分類3（10）②並列処理の隘路・（11）マルチプロセッサシステムの用語例／TOP500「November 2025」／Wikipedia「Symmetric multiprocessing」・「Non-uniform memory access」

# 量子コンピュータは「0と1を重ね合わせた量子ビット」で計算する

| | 量子ゲート方式 | 量子アニーリング方式 |
|---|---|---|
| 計算のしかた | 量子ビットにゲート操作を順に加える | エネルギーが一番低い状態を探す |
| 向く問題 | 汎用（加算も、素因数分解なども） | 組合せ最適化問題に特化 |
| 動く環境 | 多くは絶対零度近くの極低温（量子の状態が壊れやすいため） | |

**量子ビット**：0と1を同時に重ね合わせてもつ。n個で $2^n$ 通りの状態を同時に扱える。観測すると、どちらかに確率的に決まる。

**R6春 問9：量子ゲート方式の説明**

| | | |
|---|---|---|
| ア | 2進数で演算・出力 | 従来のコンピュータ |
| イ | 特定のアルゴリズムだけ、加算できない | アニーリング方式に近い |
| **ウ** | **量子ビットと重ね合わせを利用** | **正解** |
| エ | 100℃以上の高温で動作 | 逆。極低温で動かす |

**R6秋 問9：量子超越性とは**

従来のコンピュータでは**実用的な時間で処理できない計算**を、量子コンピュータが高速に実行できること（**ウ**）。「量子力学で作られている」「小さい」ではない。



詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 1-2 応用数学

# 量子ゲート方式は「汎用の回路」、アニーリング方式は「最適化専用」

| | 量子ゲート方式 | 量子アニーリング方式 |
|---|---|---|
| しくみ | 量子ビットに「ゲート」を順にかけて回路を作る（古典の論理回路の量子版） | 問題をエネルギーの形に置き換え、量子の揺らぎを弱めながら一番低い状態に落ち着かせる |
| 得意 | 汎用（素因数分解・量子化学・探索など） | 組合せ最適化（配送ルート・割当て） |
| 実例 | Google の Sycamore（2019）、IBM の量子コンピュータ | D-Wave の量子アニーリングマシン |

**D-Wave の説明：**アニーリング型は「エネルギーの低い解」を自然に返す。最適化問題は一番低い解（最小値）が欲しく、サンプリングの問題は低い解を多く集めたい。

| 押さえる3つの言葉（IBM の説明より） | |
|---|---|
| **重ね合わせ** | 0と1の両方の状態を同時にとる。測ると0か1に決まる |
| **もつれ** | 複数の量子ビットの状態が結び付き、1つを測ると他の状態も分かる |
| **干渉** | 答えの確率を波のように強め合い・打ち消し合いさせ、正解を出やすくする |

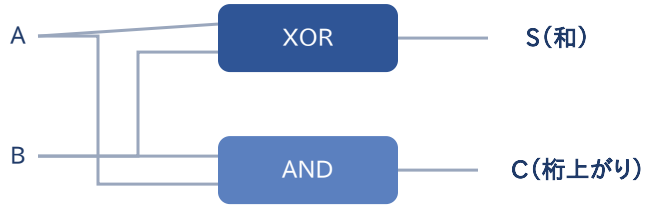
## 量子超越性（Nature, 2019）

Google は 54量子ビット（うち動いたのは53）の Sycamore で、乱数の回路の出力を集める課題を約200秒で行い、当時のスーパーコンピュータなら約1万年かかると主張した。**「古典コンピュータでは現実的な時間で解けない計算を量子で解いた」**ことが量子超越性。

**過去問：**R6春 問9（ゲート方式）・R6秋 問9（量子超越性）。

出典：IPA 令和6年度春期・秋期 午前 問9／Arute et al.「Quantum supremacy using a programmable superconducting processor」Nature 574 (2019)／IBM「What is quantum computing?」／D-Wave「What is Quantum Annealing?」

# 足し算は論理回路の組合せ。符号付きの加算は「同じ符号どうし」であふれる




半加算器（下からの桁上がりは受けない）。2つと OR で全加算器

| 回路 | 1を出す条件 |
|---|---|
| AND／OR／NOT | 両方1／どちらか1／入力が0 |
| NAND／NOR | 両方1以外／両方0（どちらか1種類で全回路を作れる） |
| XOR | 2つが異なる |

| R3秋 問8：16ビット符号付き加算がオーバーフローしないもの | | |
|---|---|---|
| ア | \|x1\|+\|x2\|≦32,768 | 16,384+16,384=32,768 は最大値32,767を超える |
| イ | ともに32,767未満 | 30,000+30,000 であふれる |
| ウ | x1×x2＞0（同じ符号） | 同じ符号どうしこそあふれる |
| **エ** | **符号が異なる** | **正解**。結果は2数の間に収まる |

16ビットの2の補数で表せるのは **−32,768〜32,767**。負の側が1つ多い。アは「≦32,768」の1つ分の差を突いた選択肢。

# 🧩 全加算器は「半加算器2つ+OR」。真理値表から組み立てる

**半加算器（1桁の足し算）**

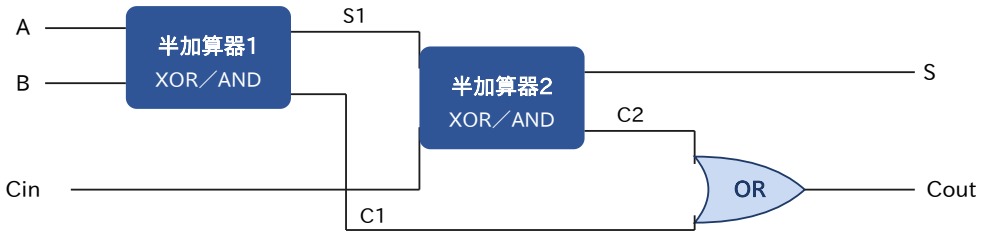
| A | B | 和 S | 桁上げ C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

S は「片方だけ1」＝XOR、C は「両方1」＝AND。

**覚え方：**和は XOR、桁上げは AND。全加算器の桁上げ出力だけは2つの桁上げを OR でまとめる。

**例題の答え：**1101＝13。桁上げが下から順に伝わるので、桁が多いほど遅い（だから実物は桁上げを先読みする回路を使う）。

**全加算器（下の桁からの桁上げ Cin も足す）**




**例題** 4ビットの 0110（6）＋ 0111（7）を、下の桁から全加算器でつないで計算（リプルキャリー）。

| 桁 | A | B | Cin | S | Cout |
|---|---|---|---|---|---|
| 0桁目 | 0 | 1 | 0 | 1 | 0 |
| 1桁目 | 1 | 1 | 0 | 0 | 1 |
| 2桁目 | 1 | 1 | 1 | 1 | 1 |
| 3桁目 | 0 | 0 | 1 | 1 | 0 |

出典：IPA シラバス Ver.7.2 中分類3（6）①演算の仕組み（半加算器・全加算器・組合せ回路）／Wikipedia「Adder (electronics)」。例題はClaudeが作った

# 2の補数のあふれは「同じ符号を足したのに、答えの符号が変わる」

| 8ビットの2の補数で表せる範囲：−128〜+127 | | |
|---|---|---|
| 計算 | 2進数 | 結果 |
| 100＋50 | 01100100＋00110010＝1 0010110 | **−106（あふれ）** |
| −100＋(−50) | 10011100＋11001110＝(1)0 1101010 | **＋106（あふれ）** |
| 100＋(−50) | 01100100＋11001110＝(1)00110010 | ＋50（正しい） |

**判定のしかた：**正＋正なのに負になった、負＋負なのに正になったら**あふれ（オーバーフロー）**。正＋負は絶対にあふれない。一番上の桁から出た(1)は捨ててよい。

**R3秋 問8の型** 16ビットの符号付き整数 x1, x2 を足して、あふれないのはどんなときか。

1. 16ビットの範囲は $-2^{15}$〜$2^{15}-1$ ＝ **−32,768〜32,767**
2. 符号が違う2数（正と負）は、足すと絶対値が小さくなる → **必ず範囲内**
3. 同じ符号どうしは、和が範囲を超えるとあふれる

**nビットの範囲 ＝ $-2^{n-1}$ 〜 $2^{n-1}-1$**

負の数の作り方：ビットを全部反転して1を足す。50=00110010 → 11001101 → 11001110（−50）。

出典：IPA 令和3年度秋期 応用情報技術者試験 午前 問8／Wikipedia「Two's complement」。8ビットの例はClaudeが計算した

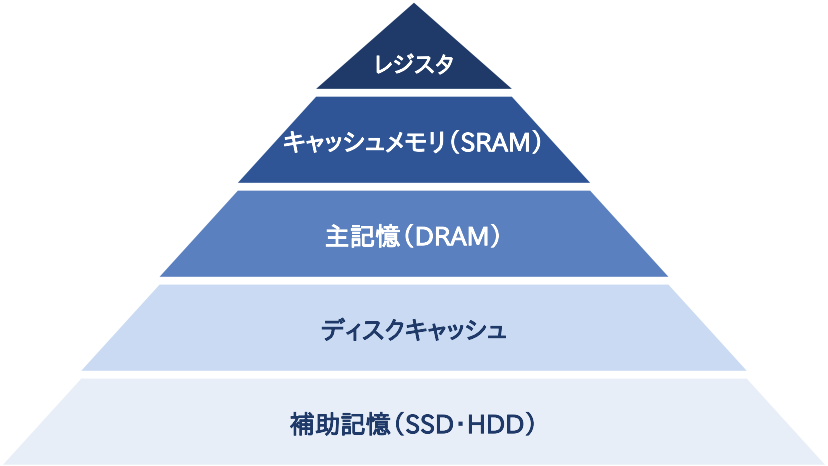
第2章

# メモリ

記憶階層 ／ RAMとROM ／ フラッシュメモリ ／ キャッシュの計算・書込み方式・対応付け ／ インタリーブ



# 記憶階層：上ほど速く小さく高い。差は「間に置く速い記憶」で埋める




| 間に置くもの | どことどこの差を埋めるか |
|---|---|
| キャッシュメモリ | プロセッサ ↔ 主記憶 |
| ディスクキャッシュ | 主記憶 ↔ 磁気ディスク |

**うまくいく理由（局所性）：**一度使ったデータはすぐまた使い（時間的）、その近くのデータも使う（空間的）。だから小さな写しでも当たる。

R3秋 問9の誤りの選択肢「主記憶と磁気ディスク装置との間にバッファメモリを置いて速度差を補う」はディスクキャッシュの説明。置き場所で見分ける。

# ⏳ 記憶階層を実測値で見る：1段下りるたびに数倍〜数千倍遅い

| 段 | 容量の例 | 待ち時間 | 1サイクルを1秒に引き伸ばすと |
|---|---|---|---|
| レジスタ | 約2KB | 0.25ナノ秒 | 1秒 |
| L1キャッシュ | 32KB | 1ナノ秒 | 4秒 |
| L2キャッシュ | 1MB | 3.5ナノ秒 | 14秒 |
| L3キャッシュ | 16〜32MB | 11.75ナノ秒 | 47秒 |
| **主記憶（DDR5）** | 64GB | **82.5ナノ秒** | **約5分半** |
| SSD（NVMe） | 2TB | 0.2ミリ秒 | 約9日 |
| HDD | 18TB | 4.16ミリ秒 | 約半年 |
| テープライブラリ | エクサバイト級 | 数分 | 数千年 |

**読み方：**4.0GHz の CPU（AMD Zen 4）で測った例。主記憶は L1の約80倍、HDD は主記憶の約5万倍遅い。だから「よく使うものを上の段に置く」キャッシュの効果が大きい。

| キャッシュが効く理由＝局所性 | |
|---|---|
| **時間的局所性** | 使ったデータはすぐまた使われる（ループの変数） |
| **空間的局所性** | 近くの番地もすぐ使われる（配列を順に読む）→ ブロック単位で運ぶ |

**試験では：**「上ほど速く・小さく・ビット単価が高い」の順番と、ディスクキャッシュ（主記憶の一部をディスクの手前に使う）とキャッシュメモリ（CPUと主記憶の間）の区別が問われる。

出典：Wikipedia「Memory hierarchy」の表（AMD Zen 4・4.0GHz換算、主記憶は DDR5-6000、SSD は Samsung 960 Pro、HDD は Seagate Exos 2X18 の値）。「引き伸ばすと」の列はClaudeが0.25ナノ秒＝1秒として換算した

# RAMは電源を切ると消え、ROMは消えない。DRAMは「リフレッシュが要る」

| 種類 | 電源を切ると | 記憶のしくみ | 特徴・用途 |
|---|---|---|---|
| DRAM | 消える | コンデンサの電荷。漏れるので一定時間ごとに書き直す（**リフレッシュ**） | 安く大容量 → **主記憶** |
| SRAM | 消える | フリップフロップ。リフレッシュ不要 | 速いが高い → **キャッシュ** |
| SDRAM・DDR | 消える | DRAMをバスのクロックに同期。DDRはクロックの立上り・立下りの2回送る | DDR2〜DDR5は差し替え不可 |
| マスクROM | 消えない | 製造時に書き込む | 書き換え不可。大量生産向け |
| PROM／EPROM | 消えない | 1回だけ書ける／紫外線で消して書き直す | — |
| EEPROM | 消えない | 電気で消して書き直す（バイト単位） | 設定値の保存 |
| **フラッシュメモリ** | 消えない | 電気で書き換え。**ブロック単位**でまとめて消す | SSD・USBメモリ・SDカード |
| FeRAM | 消えない | 強誘電体 | 速い・低電力・書換え回数が多い |

> D（Dynamic）＝記憶を保つのに動作が要る、S（Static）＝何もしなくても保つ。「リフレッシュ不要で高速」はSRAMの方。

# DRAMは「コンデンサ1つ」、SRAMは「トランジスタ6つ」。作りの差が性質の差

DRAM の1ビット




電荷が少しずつ漏れる
→ 定期的に読んで書き直す
（リフレッシュ）

SRAM の1ビット




フリップフロップで値を保つ
→ リフレッシュ不要・速い
ただし1ビットが大きい

| | DRAM | SRAM |
|---|---|---|
| 速さ | 遅め | 速い |
| 集積度・ビット単価 | 高い・安い | 低い・高い |
| リフレッシュ | **要る**（各行を64ミリ秒以内に。DDR5は32ミリ秒） | 要らない |
| 用途 | 主記憶 | キャッシュメモリ |

| 主記憶の規格 DDR4 と DDR5 | DDR4 | DDR5 |
|---|---|---|
| 電圧 | 1.2V | 1.1V |
| 標準の速さ | 1,600〜3,200 MT/s | 4,000〜6,400 MT/s |
| 1枚の中のチャネル | 64ビット×1 | 32ビット×2（独立） |
| チップ内のECC | なし | あり（オンダイECC） |

**例題** DDR5-4800 のメモリ1枚（データ幅64ビット）の最大転送速度は？

1. 1秒に 4,800M回 転送（MT/s=100万回／秒）
2. 1回に 64ビット=8バイト
3. 4,800M × 8 = **38.4GB/秒**（だから「PC5-38400」と呼ぶ）

DDR はクロックの立上り・立下りの両方で送る。DIMM=デスクトップ用、SO-DIMM=ノート用。

出典：IPA シラバス Ver.7.2 中分類3 2.（1）用語例／Wikipedia「Dynamic random-access memory」（JEDEC 規格で各行64ミリ秒以内のリフレッシュ）／「DDR5 SDRAM」（1.1V、4,000〜6,400MT/s、2つのサブチャネル、オンダイECC）。「DDR4 SDRAM」（1.2V、1,600〜3,200MT/s）。例題はClaudeが計算した

# 不揮発メモリは「どうやって書き換えるか」で見分ける

| 名前 | 書き方・消し方 | 決め手のことば |
|---|---|---|
| マスクROM | 製造時に書き込む。変えられない | 製造時 |
| PROM | 利用者が1回だけ書ける | 一度だけ |
| EPROM | 紫外線を当てて全部消す | 紫外線 |
| EEPROM | 電気でバイト単位に消して書く | 電気的に |
| フラッシュ | 電気でブロック単位にまとめて消す | ブロック単位で一括消去 |
| FeRAM | 強誘電体の分極の向きで記憶 | 強誘電体 |
| MRAM | 磁気の向き（磁気トンネル接合）で記憶 | 磁気 |
| **相変化メモリ** | 熱で**結晶とアモルファス**を切り替え、抵抗の差で記憶 | 結晶／非結晶 |

**1つのセルに何ビット入れるか（NAND）**

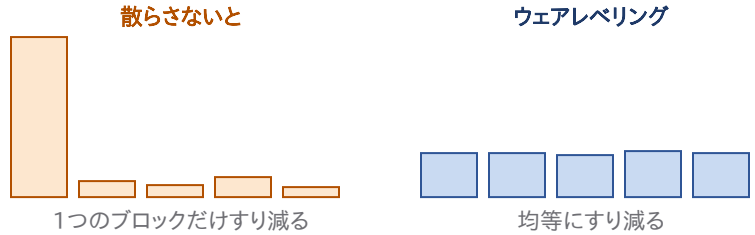
| 種類 | ビット／セル | 電圧の段階 |
|---|---|---|
| SLC | 1 | 2 |
| MLC | 2 | 4 |
| TLC | 3 | 8 |
| QLC | 4 | 16 |

段階を細かく分けるほど大容量・安価になるが、読み分けが難しくなり、書換え回数の上限と速さは下がる。キオクシア（旧東芝メモリ）は2001年に MLC の NAND を世界で初めて製品にした。

**過去問：**H30秋 問10（相変化メモリ）の誤りは、PROM・SRAM・DRAM の説明。

出典：IPA シラバス Ver.7.2 中分類3 2.（1）用語例・IPA 平成30年度秋期 午前 問10／キオクシア「SLC NAND」（多値化技術）／Wikipedia「Phase-change memory」・「Ferroelectric RAM」・「Flash memory」

# フラッシュメモリは書換え回数に上限。だから書込みを散らして寿命を延ばす



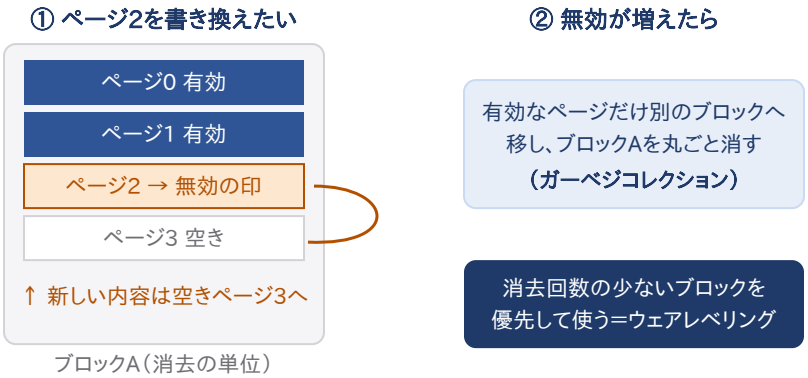

| 用語 | 何か |
|---|---|
| ウェアレベリング | 書換え回数を管理し、書く位置を散らす**技術** |
| ライトアンプリフィケーション | 少し書くだけで多く書き直してしまう**現象**（寿命が縮む） |
| 不良ブロックの代替 | 壊れたブロックを予備で置き換える |
| 多値セル（MLC・TLC） | 1セルに複数ビット。大容量だが寿命は短め |

R5春 問11・R5秋 問10はどちらも答えがウェアレベリング（ア）。誤りの選択肢は上の表の残り（とジャーナリング・デフラグ）。

| R6春 問11：圧縮したプログラムを主記憶に展開する時間 | |
|---|---|
| 圧縮後の大きさ | 15Mバイト × 40% ＝ 6Mバイト |
| 転送 | 6 ÷ 20Mバイト/秒 ＝ 0.3秒 |
| 展開（圧縮後1Mバイトあたり0.03秒） | 6 × 0.03 ＝ 0.18秒 |
| **合計（同時には行わない）** | **0.48秒（ア）** |

誤りの選択肢は「圧縮前の15Mバイトで計算した」もの。0.75（15÷20だけ）、0.93（0.75+0.18）、1.20（0.75+15×0.03）。**どの大きさで計算するか**を問題文で確かめる。

# NANDは「ページで書き、ブロックで消す」。だから上書きできない




OS がファイルを消したことを SSD に伝える命令が TRIM。どのページが要らないか分かるので、ガーベジコレクションの無駄が減る。

| | NAND型 | NOR型 |
|---|---|---|
| 読み書きの単位 | ページ単位 | バイト単位で読める |
| 消去の単位 | ブロック（数KB〜数MB）単位 | |
| 得意 | 大容量・順に読む | プログラムをそのまま実行 |
| 用途 | SSD、SDカード、USBメモリ | 機器の設定・起動プログラム |

**H29春 問11＝R6春 問11** 15MB のプログラムを40%に圧縮してフラッシュに格納。転送 20MB/秒、1MB の展開に 0.03秒。転送と展開は同時に行わない。

1. 圧縮後：15 × 0.4 ＝ 6MB
2. 転送：6 ÷ 20 ＝ 0.3秒
3. 展開：6 × 0.03 ＝ 0.18秒 → 合計 **0.48秒**

出典：IPA 平成30年度春期 午前 問10（NAND＝ページ単位）・平成29年度春期 午前 問11・令和6年度春期 午前 問11・令和5年度春期 問11・秋期 問10（ウェアレベリング）／Wikipedia「Flash memory」（ブロックは4KB〜4MB程度）・「Wear leveling」・「Trim (computing)」

# 📦 SDカードは「容量の規格」と「速さの規格」を分けて読む

| 容量の規格（SD Association） | | |
|---|---|---|
| **規格** | **容量** | **ファイルシステム** |
| SD | ～2GB | FAT12／16 |
| SDHC | 2GB超～32GB | FAT32 |
| **SDXC** | **32GB超～2TB** | **exFAT** |
| SDUC | 2TB超～128TB | exFAT |

R1秋 問12：SDXC＝exFAT・最大2TB（エ）。誤りは SDIO（カード形の周辺機器）、microSDHC（小型・32GB）、AACS（Blu-ray の著作権保護。SD は CPRM）。

| 速さ（バスの規格） | | |
|---|---|---|
| **バス** | **最大の速さ** | **しくみ** |
| Default（SD1.0） | 12.5MB/秒 | — |
| High Speed（SD1.1） | 25MB/秒 | デジタルカメラ向け |
| UHS-I | 104MB/秒 | 1列のピン・1.8V |
| UHS-II／III | 312／624MB/秒 | 2列目のピンで2レーン |
| SD Express | 3,940MB/秒 | PCIe Gen4 ＋ NVMe |

容量の規格と速さの規格は別。「SDXC で UHS-I」のように組み合わせて表示する。速いバスは SDHC・SDXC・SDUC のどれにも使える。

出典：IPA 令和元年度秋期 午前 問12／SD Association「Capacity (SD/SDHC/SDXC/SDUC)」・「Bus Speed」／Wikipedia「SD card」（UHS-I 104MB/秒、1.8V）

# ⏱️ キャッシュの計算は1つの式：当たった分＋外れた分

**実効アクセス時間 ＝ h × キャッシュの時間 ＋ (1 − h) × 主記憶の時間**

h=ヒット率

## R4春 問9＝R6秋 問10

キャッシュは主記憶の1/30、h=95%

| | |
|---|---|
| 当たった分 | 0.95 × 1/30 ≒ 0.032 |
| 外れた分 | 0.05 × 1 ＝ 0.05 |
| **合計** | **約0.08倍（イ）** |

0.03 は全部当たったときの値、0.95 はヒット率そのもの。

## R6春 問10：ヒット率を逆算

主記憶60ns、キャッシュ10ns、実効15ns

| | |
|---|---|
| 式 | 10h ＋ 60(1−h) ＝ 15 |
| 整理 | 60 − 50h ＝ 15 |
| **答え** | **h ＝ 0.9（エ）** |

0.1 はミス率。0.17 は10÷60で、時間の比を答えた誤り。

## R4秋 問10：2段キャッシュ

L1のヒット率0.95、L2のヒット率0.6

| | |
|---|---|
| L1で当たる | 0.95 |
| L1で外れL2で当たる | 0.05 × 0.6 ＝ 0.03 |
| **合計** | **0.98（エ）** |

0.57（0.95×0.6）は「両方に当たる確率」で誤り。

# ⏱ 実効アクセス時間は2つの書き方がある。どちらも同じ答えになる

**書き方1：重み付き平均（試験の定番）**

**h × キャッシュ ＋ (1−h) × 主記憶**

**書き方2：ミスしたら追加で待つ**

**キャッシュ ＋ (1−h) × (主記憶 − キャッシュ)**

**H21秋 問11** キャッシュ10ナノ秒、主記憶60ナノ秒、ヒット率90%。
書き方1：0.9×10+0.1×60=9+6=**15**　書き方2：10+0.1×50=**15**

書き方2は「ヒットしてもしなくてもまずキャッシュを見る」考え方。問題文に「キャッシュを見てから主記憶を読む」とあれば、10+0.1×60=16 のように足し方が変わるので条件を読む。

**R1秋 問10** キャッシュ容量 a、時間 x。主記憶容量 b、時間 y。キャッシュに**ない**確率 r。平均アクセス時間の式は？

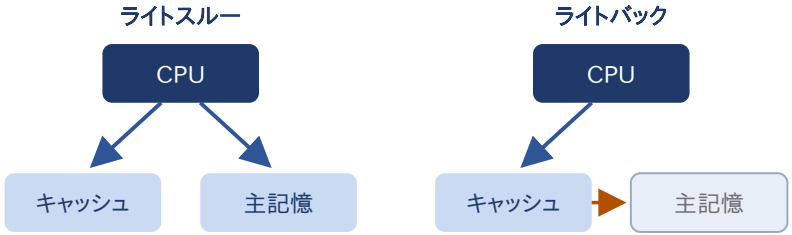
1. r は「ない」確率＝ミス率。ヒット率は 1−r
2. 容量 a・b は式に入らない（じゃまの数）
3. (1−r)x ＋ ry（**イ**）

**この型の出題（9回）**

| | |
|---|---|
| 数値を入れる | H21秋・H22秋・H25秋・H29秋、R4春 問9 |
| ヒット率を逆算 | R6春 問10 |
| 2段キャッシュ | R4秋 問10 |
| 記号の式 | R1秋 問10 |

出典：IPA 平成21年度秋期 問11・令和元年度秋期 問10 ほか表の回（午前）／Wikipedia「CPU cache」。書き方2との対応はClaudeが式を変形して確かめた

# ライトスルーは「両方に書く」、ライトバックは「追い出すとき書き戻す」




| R3春 問12=R5春 問10（ほぼ同じ）、R4秋 問9 | |
|---|---|
| **選択肢の言い方** | **どちらの説明か** |
| キャッシュだけに書き込むので高速 | ライトバック |
| 両方に同時に書き込むので、主記憶の内容は常に最新 | **ライトスルー（正解）** |
| 追い出す時に主記憶のデータを変更する | ライトバック |
| 主記憶へのアクセスが少なく、バスの占有率が低い | ライトバック |
| 追い出すときに書き戻す必要が生じることはない | 誤り（ライトバックは必要） |

> 誤りの選択肢の多くは、2つの方式の説明を**入れ替えた**もの。「スルー＝通り抜けて主記憶まで」と覚える。

# ライトバックの動きを追う：3回書いても主記憶に書くのは1回

| # | CPUの操作 | キャッシュ（X） | ダーティ | 主記憶（X） | 主記憶へ書く回数 WB／WT |
|---|---|---|---|---|---|
| 1 | X を読む（ミス） | 5 を持ってくる | 0 | 5 | 0／0 |
| 2 | X に 6 を書く | 6 | **1** | 5（古いまま） | 0／1 |
| 3 | X に 7 を書く | 7 | 1 | 5 | 0／2 |
| 4 | X に 8 を書く | 8 | 1 | 5 | 0／3 |
| 5 | X が追い出される | — | — | **8 に書き戻す** | **1／3** |

**ダーティビット：**「キャッシュだけ書き換わって、主記憶はまだ古い」という印。追い出すとき、印が1のブロックだけ書き戻す。印が0なら捨てるだけでよい。

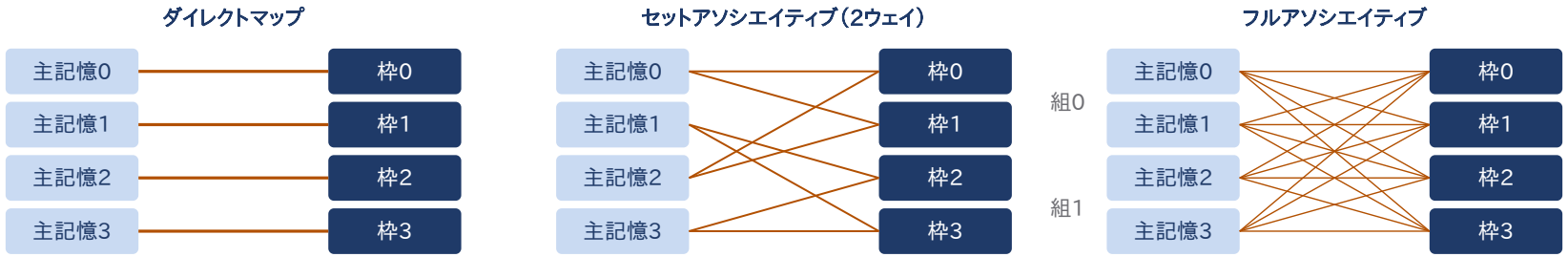
| | ライトスルー | ライトバック |
|---|---|---|
| 書く先 | 両方に毎回 | キャッシュだけ |
| 主記憶への書込み | 多い | **少ない** |
| 一致（一貫性） | いつも一致 | しばらくずれる |
| 追い出し時 | 捨てるだけ | 書き戻しが要る |

**9回出た：**目的を問う（H25春・H26秋＝書込み頻度を減らす）、説明を選ぶ（H29春・R3春・R4秋）など。「一貫性が保たれる」「書き戻し不要」はライトスルーの側。

マルチコアでは、ほかのコアのキャッシュとのずれを見張る仕組み（スヌープ方式など）でつじつまを合わせる。

出典：IPA 平成25年度春期 問12・平成26年度秋期 問9・平成29年度春期 問10・令和3年度春期 問12・令和4年度秋期 問9（午前）／Wikipedia「CPU cache」・「Cache coherence」。表の値はClaudeが作った例

# キャッシュの対応付けは「置ける場所が1か所・決まった組の中・どこでも」




| 方式 | 良いところ | 弱いところ |
|---|---|---|
| ダイレクトマップ | 探すのは1か所で速い・簡単 | 同じ枠を取り合って追い出し合う |
| セットアソシエイティブ | 2つの中間（nウェイ） | — |
| フルアソシエイティブ | 空いていればどこでも置ける | 全部を同時に比べる回路（連想メモリ）が要る |

R4春 問10：フルアソシエイティブは「任意のキャッシュのブロックを、主記憶のどの部分にも割り当てられる」（エ）。「ブロックが固定されている」はダイレクトマップ、「セットが固定」はセットアソシエイティブ寄りの説明。

満杯のときに追い出すのは、ふつうLRU（最後に使ってから一番長いもの）。

# ダイレクトマップは番地を3つに切る：タグ・インデックス・オフセット

**例題** 主記憶の番地は16ビット。1ブロック＝16バイト、キャッシュは8ブロック（ダイレクトマップ）。番地 **0x1234** はキャッシュのどこに入るか。

| タグ 9ビット | インデックス 3ビット | オフセット 4ビット |
|---|---|---|
| 0001 0010 0 | 011 | 0100 |
| ＝ 36（0x024） | ＝ 3番の枠 | ＝ ブロックの4バイト目 |
| 枠に入っているのが誰かの名札 | $2^3$=8枠 | $2^4$=16バイト |

1. 0x1234 を2進数に：0001 0010 0011 0100
2. 下から4ビットがオフセット、次の3ビットがインデックス、残り9ビットがタグ
3. 3番の枠を見て、名札（タグ）が36ならヒット、違えばミス

**衝突の例**

| 番地 | インデックス | タグ |
|---|---|---|
| 0x1234 | 3 | 36 |
| 0x1A34 | 3 | 52 |

2つを交互に使うと、同じ3番の枠を取り合って毎回ミスする（競合ミス）。2ウェイのセットアソシエイティブなら同じ組に2つとも置けるので、両方ヒットする。

**過去問：**H21春 問13（複数の特定ブロック＝セットアソシエイティブ）、H21秋 問10・H24春 問10（単一のロケーション＝ダイレクトマップ）、R4春 問10（任意＝フルアソシエイティブ）。

出典：IPA 平成21年度春期 問13・平成21年度秋期 問10・平成24年度春期 問10・令和4年度春期 問10（午前）／Wikipedia「CPU cache」（タグ・インデックス・オフセット）。例題はClaudeが作った

# 追い出す相手の選び方：LRU と FIFO を手で追う

**例題** 3つの枠に入るキャッシュ（どこにでも置ける）。A B C A D A B の順に使う。最初は空。ヒットは何回か。

**LRU（最後に使ってから一番長いものを出す）**

| 使う | 結果 | 中身（古い→新しい使用順） |
|---|---|---|
| A | ミス | A |
| B | ミス | A B |
| C | ミス | A B C |
| A | **ヒット** | B C A |
| D | ミス（Bを出す） | C A D |
| A | **ヒット** | C D A |
| B | ミス（Cを出す） | D A B |

**ヒット2回。**よく使う A が残り続ける。

**FIFO（入れたのが一番古いものを出す）**

| 使う | 結果 | 中身（入れた順） |
|---|---|---|
| A | ミス | A |
| B | ミス | A B |
| C | ミス | A B C |
| A | **ヒット** | A B C（順番は変わらない） |
| D | ミス（Aを出す） | B C D |
| A | ミス（Bを出す） | C D A |
| B | ミス（Cを出す） | D A B |

**ヒット1回。**使ったかどうかを見ないので、よく使う A を追い出してしまう。

出典：Wikipedia「Cache replacement policies」。同じ考え方は仮想記憶のページ置換え（#5 ソフトウェア）でも計算問題になる。例題はClaudeが作った

# 📦 主記憶の3つの工夫:インタリーブ・エンディアン・ユニファイドメモリ

## メモリインタリーブ

| バンク0 | バンク1 | バンク2 | バンク3 |
|---|---|---|---|
| 0番地 | 1番地 | 2番地 | 3番地 |
| 4番地 | 5番地 | 6番地 | 7番地 |
| 8番地 | 9番地 | 10番地 | 11番地 |

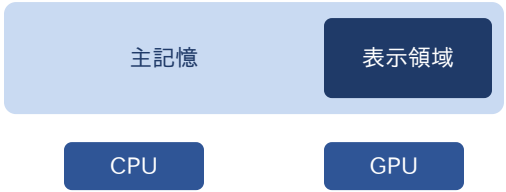
連続した番地を**別々のバンク**に置き、同時に読み書きを進める。R3秋 問9の正解(エ)。誤りは仮想記憶・ディスクキャッシュ・DMAの説明。

## エンディアン(12345678 を1000番地から)

| 番地 | 1000 | 1001 | 1002 | 1003 |
|---|---|---|---|---|
| ビッグ | **12** | 34 | 56 | 78 |
| リトル | **78** | 56 | 34 | 12 |

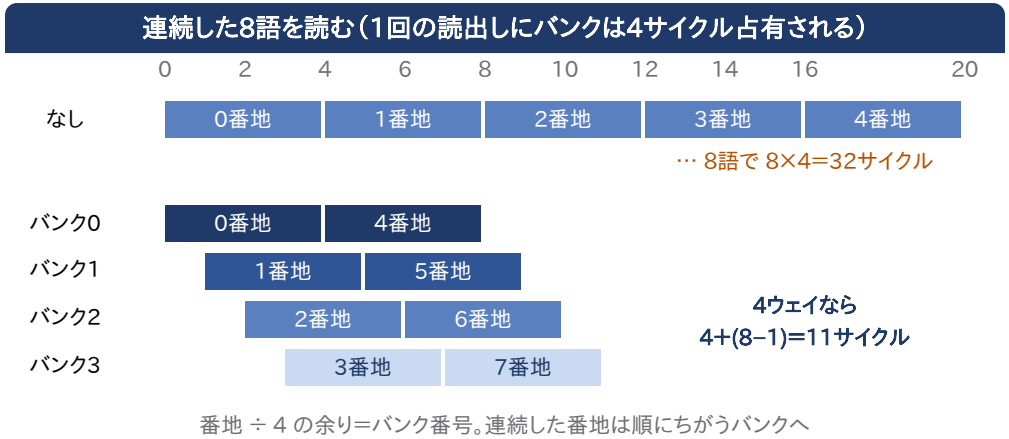
**ビッグ**=上位から小さい番地へ(人が読む順・ネットワーク)。**リトル**=下位から(x86系のCPU)。

## ユニファイドメモリ




**主記憶の一部**を表示用に使い、CPUとGPUで共有。R5秋 問11=R7秋 問9の正解(イ)。「専用のフレームバッファをもつ」は逆。

# インタリーブはバンクの「準備時間」を重ねる。8回出た最頻出テーマ




**考え方はパイプラインと同じ:** 1つのバンクが準備している間に、次のバンクに読出しを始めさせる。最初の1語は4サイクル、あとは1サイクルごとに届く。

| 誤りの選択肢 | 何の説明か |
|---|---|
| 主記憶と外部記憶を一元的にアドレス付け | 仮想記憶 |
| CPUと主記憶の間に高速の記憶を置く | キャッシュメモリ |
| CPUと磁気ディスクの間にバッファ | ディスクキャッシュ |
| 1つのバンクの故障に備える | ミラーリング・ECC |

H21春・H23秋・H25秋・H28秋・H29秋・H30春・R2秋・R3秋 問9。正解はいつも「複数のバンクに分けて並行してアクセス」。

出典:IPA 応用情報技術者試験 午前（表に挙げた8回）／Wikipedia「Memory interleaving」。サイクル数の例はClaudeが作った

# ハミング符号で1ビットの誤りを直してみる

**例題** データ 1011 に検査ビット3つを付けて7ビットにする（ハミング(7,4)符号）。送る途中で5ビット目が反転した。

| 位置 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| 役割 | p1 | p2 | d1 | p4 | d2 | d3 | d4 |
| 送った | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 届いた | 0 | 1 | 1 | 0 | 1 | 1 | 1 |

1. p1 は位置1・3・5・7、p2 は2・3・6・7、p4 は4・5・6・7 の1の数を偶数にする検査ビット
2. 届いた側で検査し直す：1・3・5・7 → 0+1+1+1＝奇数（×）、2・3・6・7 → 偶数（○）、4・5・6・7 → 奇数（×）
3. ×の検査ビットの番号を足す：1+4=**5** → 5ビット目が誤り。反転すれば直る

| 方式 | 検出 | 訂正 |
|---|---|---|
| パリティ（奇数・偶数） | 1ビット | できない |
| 水平＋垂直パリティ | 1ビット | 1ビット |
| チェックサム | できる | できない |
| CRC | 連続した誤りに強い | できない |
| **ハミング符号**＋全体パリティ | **2ビット** | **1ビット** |

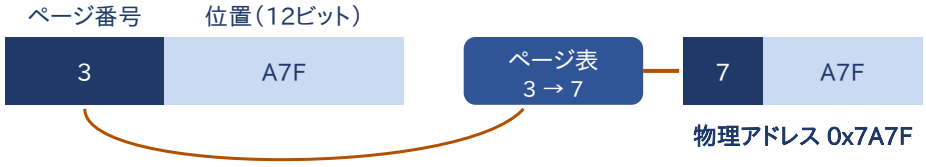
**ECCメモリ：**64ビットのデータに8ビットの検査ビットを足し、72ビットで記憶する（SECDED＝1ビット訂正・2ビット検出）。サーバの主記憶で使う。

**4回とも同じ問い：**「2ビット検出・1ビット訂正」はどれか（H21春・H22春・H30秋・R2秋）。答えはハミング符号。

出典：IPA 平成21年度春期 問12・平成22年度春期 問11・平成30年度秋期 問9・令和2年度10月 問10（午前）／Wikipedia「Hamming code」・「ECC memory」・「DDR5 SDRAM」（ECCなし64本・ECCあり72本のデータ線）。例題はClaudeが計算した

# MMU は仮想アドレスを物理アドレスに変える「番地の翻訳係」

**例題** ページの大きさ 4KB（＝$2^{12}$バイト）。仮想アドレス **0x3A7F** は物理アドレスのどこか。ページ表では、ページ3 → ページ枠7。




ページ内の位置はそのまま。ページ番号だけを置き換える

ページ表は主記憶にあるので毎回引くと遅い。最近の変換結果を覚えておく小さな速い記憶が **TLB**。ページが主記憶になければ**ページフォールト**を起こし、OS がディスクから読み込む。

**H27秋 問10「MMUの説明」**

| 選択肢の中身 | 本当は |
|---|---|
| **仮想アドレスを物理アドレスに対応させる** | **正解** |
| ページフォールトを受けてスワップする | OS の仕事 |
| OS の一部でキャッシュ制御・バス調整 | MMU はハードウェア |
| CPU と主記憶の速度差を吸収 | キャッシュメモリ |

仮想記憶のしくみ（ページング・置換え）は #5 ソフトウェアで、翻訳するハードウェア（MMU）がこの分野。

出典：IPA 平成27年度秋期 応用情報技術者試験 午前 問10／Wikipedia「Memory management unit」・「Virtual memory」。例題はClaudeが作った

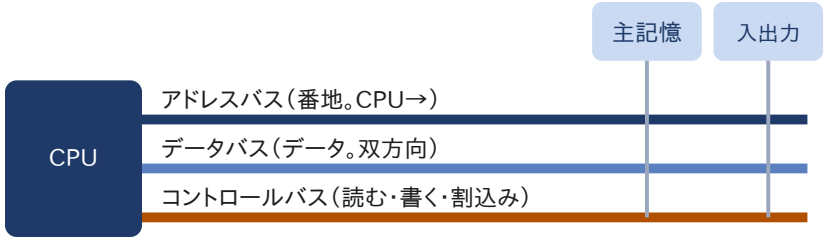
第3章

# バスと入出力デバイス

バスの種類と転送能力 ／ バスの調停 ／ 有線・無線のインタフェース ／ 接続形態 ／ DMA



# バスは「番地・データ・指示」の3種類の線。転送能力は「幅×周波数」




アドレスバス n本 → $2^n$ 番地まで指定できる（32本なら4Gバイト）

**転送能力 ＝ 幅（バイト）× クロック周波数 × 転送回数／クロック**

64ビット・200MHz・1回 → 8バイト × 2億 ＝ 1.6Gバイト/秒

| バスの調停 | やり方 |
|---|---|
| 集中制御方式 | 1つの調停回路（バスアービタ）が一括して決める |
| デイジーチェーン | 数珠つなぎ。調停回路に近い装置ほど優先 |
| ポーリング方式 | 制御する側が順に問い合わせる |
| 割込み方式 | 装置の側から合図して知らせる |

バスを使う権利をもって自分から転送を始める装置が**バスマスタ**（CPU・DMAコントローラ）。

計算の誤りは**ビットとバイト**（8倍の差）と、**DDR**（1クロックに2回）の取り違え。



詳しく：応用情報.com 用語集 3-3 バス ／ 応用情報.com 用語集 3-1 プロセッサ

# 🚌 転送速度は「1回に運ぶ量 × 1秒間の回数」。3つの例で練習

| H31春 問10：メモリ | |
|---|---|
| 1回に運ぶ量 | 16ビット＝2バイト |
| 1秒間の回数 | 1 ÷ 80ナノ秒 ＝ 1,250万回 |
| **答え** | **25Mバイト/秒（イ）** |

200 はビットのまま、12.5 は1バイトと取り違えた値。

| DDR5-4800 のメモリ1枚 | |
|---|---|
| 1回に運ぶ量 | 64ビット＝8バイト |
| 1秒間の回数 | 48億回（4,800MT/s） |
| **答え** | **38.4Gバイト/秒** |

「MT/s」は1秒に何百万回送るか。クロック周波数（2,400MHz）の2倍。

| PCI Express 4.0 ×16 | |
|---|---|
| 1レーン | 16GT/s、符号化 128b/130b |
| 1レーンの実効 | 16 × 128/130 ÷ 8 ≒ 1.97GB/秒 |
| **16レーン** | **約31.5GB/秒（片方向）** |

符号化の分だけ目減りする。8b/10b なら2割減。

| PCIe の世代 | 1レーンの転送 | 符号化 | ×16 の片方向 |
|---|---|---|---|
| 3.0（2010） | 8GT/s | 128b/130b | 約15.8GB/秒 |
| 4.0 | 16GT/s | 128b/130b | 約31.5GB/秒 |
| 5.0 | 32GT/s | 128b/130b | 約63GB/秒 |
| 6.0 | 64GT/s | PAM4＋FLIT | 約128GB/秒 |

| | SATA（AHCI） | NVMe |
|---|---|---|
| つなぐ線 | SATA 6Gbit/秒 | PCI Express |
| 命令の列（キュー） | 1本・32個まで | 最大65,535本 |

NVMe は SSD のために作られた手順。たくさんの列で並行して命令を受けられる。

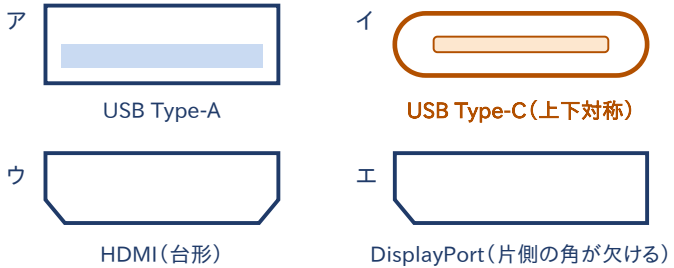
出典：IPA 平成31年度春期 午前 問10／Wikipedia「PCI Express」（世代ごとの GT/s・符号化・×16 の帯域）／NVM Express「About」／Wikipedia「NVM Express」（最大65,535キュー）・「SATA」。計算はClaudeが行った

# 有線のインタフェースは今ほぼ全部シリアル。名前と用途の組で覚える

| 規格 | 主な用途 | 押さえどころ |
|---|---|---|
| USB | 周辺機器全般 | ハブでツリー状に127台。給電もできる |
| Thunderbolt | 高速な外付け機器 | Type-C と同じ形の端子 |
| HDMI | テレビ・ディスプレイ | 映像＋音声をデジタルで1本に |
| DisplayPort | PCのディスプレイ | ディスプレイを数珠つなぎにできる |
| シリアルATA | 内蔵HDD・SSD | パラレルATAの後継 |
| PCI Express | 拡張カード・SSD | レーンを束ねて速く（×16など）。NVMeはこの上の手順 |
| **ファイバチャネル** | サーバ⇔ストレージ（SAN） | 光ケーブルも電気ケーブルも使える |
| RS-232C | 機器の設定用 | 昔からある低速の1対1 |

R6秋 問11：FCの特徴は「電気ケーブル又は光ケーブルを用いることができる」（**ウ**）。TCP/IPの上で動くのは iSCSI、パラレルSCSIは別物、接続形態はスイッチだけに限らない。

**R3秋 問10：USB Type-C の断面（正解はイ）**




原文の図を簡略化して描き直したもの



詳しく：応用情報.com 用語集 3-4 入出力デバイス ／ 応用情報.com 用語集 3-3 バス

# USBの世代は「1レーンの速さ × レーン数」。名前が変わったので表で整理

| 規格（今の呼び方） | 最大の速さ | 通信 | 過去問 |
|---|---|---|---|
| USB 1.1 ロー／フルスピード | 1.5／12Mbps | 半二重 | — |
| USB 2.0 ハイスピード | 480Mbps | 半二重 | H21秋 問12 |
| USB 3.0 スーパースピード（3.2 Gen1） | 5Gbps | **全二重** | H28春 問10・H30春 問12 |
| USB 3.2 Gen2／Gen2×2 | 10／20Gbps | 全二重 | — |
| USB4（Thunderbolt の方式がもと） | 40Gbps → 80Gbps | 全二重 | — |

**USB-IF の説明：**USB 3.2 は以前の 3.x を吸収し、5・10・20Gbps の3つの速さを定める。20Gbps は 10Gbps のレーンを2本使う。USB4 は インテルが提供した Thunderbolt の方式をもとに、80Gbps 対応ケーブルで最大80Gbps。

| 給電（USB Power Delivery） | |
|---|---|
| USB 2.0 ／ 3.0 の標準 | 500mA ／ 900mA |
| USB PD（〜2021） | 20V×5A＝**100W** |
| USB PD 3.1 | 48V×5A＝**240W** |

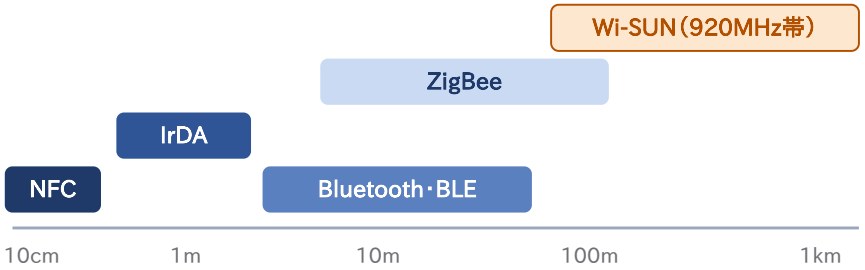
給電する向きも固定されない。モニタからノートPCに給電しながら映像を映すこともできる。

| 映像のインタフェース | |
|---|---|
| DisplayPort（VESA） | 映像と音声を**パケット**にして送る。1.4 で32.4Gbps、2.1 で最大80Gbps |
| HDMI 2.2 | 最大96Gbps（Ultra96 ケーブル） |

R1秋 問11：DisplayPort は「パケット化してシリアル伝送」。アナログは送れない。

出典：IPA 午前（表の各回・令和元年度秋期 問11）／USB-IF「USB 3.2」・「USB4」・「USB Charger (USB Power Delivery)」／VESA「About DisplayPort」／HDMI Licensing「HDMI 2.2」／Wikipedia「USB」・「DisplayPort」

# 近距離の無線は「届く距離」と「電力」で選ぶ




距離の目安。NFC＝かざす（10cm）、IrDA＝赤外線で向かい合わせ（1m以内）、Bluetooth＝身の回り、ZigBee＝センサを網目状に、Wi-SUN＝屋外まで（スマートメーター）。

| 規格 | 使い道 |
|---|---|
| NFC | 交通系ICカード・スマホ決済（13.56MHz帯） |
| Bluetooth | イヤホン・キーボード（2.4GHz帯） |
| BLE | 電池で長く動くセンサ・ビーコン |
| ZigBee | 多数のセンサを網目状に（低速・低電力） |
| Wi-SUN | スマートメーター（数百m〜1km） |
| IrDA | 昔の携帯電話の赤外線通信 |

これらはまとめて**無線PAN**（個人の周りのネットワーク）。周波数が低いほど回り込んで遠くまで届く。

# 無線の規格を、規格団体の公式の数字で比べる

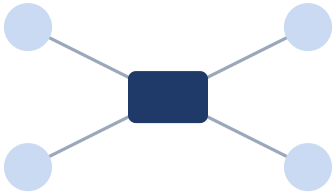
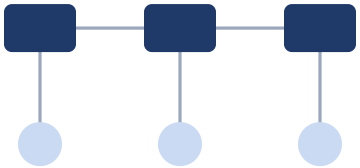
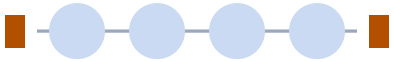
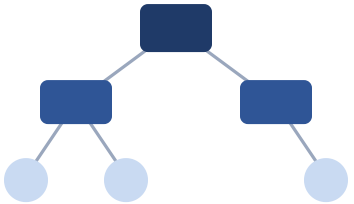
| 規格 | 周波数 | 距離・速さ | 決め手（公式の説明） | 規格団体 |
|---|---|---|---|---|
| NFC | 13.56MHz | ふつう2cmまで・46kbps〜1.7Mbps | 「かざす」のが前提。片方から電力も送れるので電池のないカードも動く。スマホがカードのふりをする「カードエミュレーション」が主な使い方 | NFC Forum |
| Bluetooth Classic | 2.4GHz帯・79チャネル | 身の回り | 1対1で音声などを流し続ける（ワイヤレスイヤホン） | Bluetooth SIG |
| Bluetooth LE | 2.4GHz帯・40チャネル | 身の回り | とても低い電力で動くように設計。メッシュネットワークや LE Audio もある | Bluetooth SIG |
| Zigbee | 2.4GHz帯・サブGHz帯 | 低速 | 自己修復するメッシュネットワーク。電池を長くもたせる（センサ・スマートホーム） | CSA |
| Wi-Fi 7 | 2.4・5・6GHz帯 | 数Gbps | 6GHz帯で320MHz幅のチャネル、複数の帯域を同時に使うマルチリンク（2024年〜） | Wi-Fi Alliance |
| LPWA | 主にサブGHz帯 | 数km〜10km超・低速 | 広い範囲・低電力・少ないデータ（スマートメーター、IoT） | （各方式） |

**H31春 問11（ZigBee）：**正解は「低消費電力・低速・センサーネットワーク」。誤りは Bluetooth・赤外線・トランシーバ。

**コツ：**近い順に NFC（cm）→ Bluetooth（m）→ Wi-Fi（建物）→ LPWA（km）。

出典：IPA 平成31年度春期 午前 問11／NFC Forum「NFC Technology」／Bluetooth SIG「Tech Overview」／CSA「Zigbee」／Wi-Fi Alliance「Wi-Fi CERTIFIED 7」／Wikipedia「LPWAN」

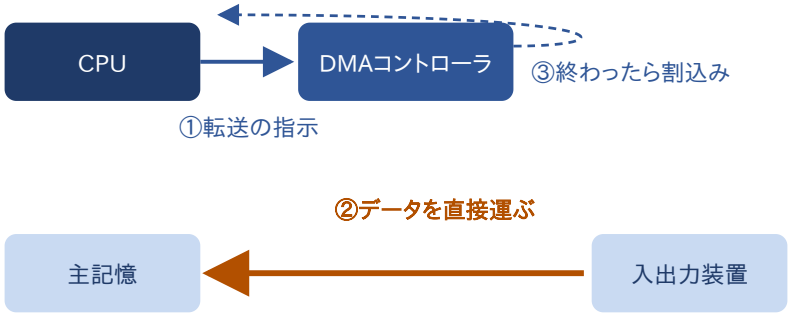
# つなぎ方は「星形・段々・数珠つなぎ・木」。送り方は今ほぼシリアル

| スター | カスケード | デイジーチェーン | ツリー |
|---|---|---|---|
|  |  |  |  |
| 中心のハブに1本ずつ。1台の故障が広がらない | ハブどうしを段々につなぐ。USBは5段まで | 数珠つなぎ。SCSIは両端に**ターミネータ** | 根元から枝分かれ。USBはこの形（最大127台） |

| | シリアル転送 | パラレル転送 |
|---|---|---|
| 送り方 | 1本の線で1ビットずつ | 複数の線で同時に |
| 速くしやすさ | しやすい（今の主流：USB・SATA・PCIe） | 線ごとの到着のずれ（**スキュー**）で限界 |

UART：並んだビットを1ビットずつの列に変えて送り、受け取った列を元に戻す回路。クロックの線を使わず、開始ビット・停止ビットで区切る（調歩同期）。

# DMAは「CPUを介さずに」主記憶と装置の間でデータを運ぶ




| 方式 | 運ぶのは | CPUの負担 |
|---|---|---|
| プログラム制御 | CPUが1データずつ | 大きい |
| DMA | DMAコントローラ | 指示と終わりの割込みだけ |
| チャネル制御 | 入出力専用のプロセッサ（大型機） | さらに小さい |

| R7春 問9：DMAコントローラーの説明 | | |
|---|---|---|
| ア | 時間のかかる積和演算を高速に | DSP |
| イ | 仮想メモリ・メモリ保護などの管理 | MMU（メモリ管理ユニット） |
| ウ | クロックを数えて時間の経過を保持 | タイマ |
| **エ** | **メモリと入出力装置、メモリ間の転送をMPUを介さずに** | **正解** |

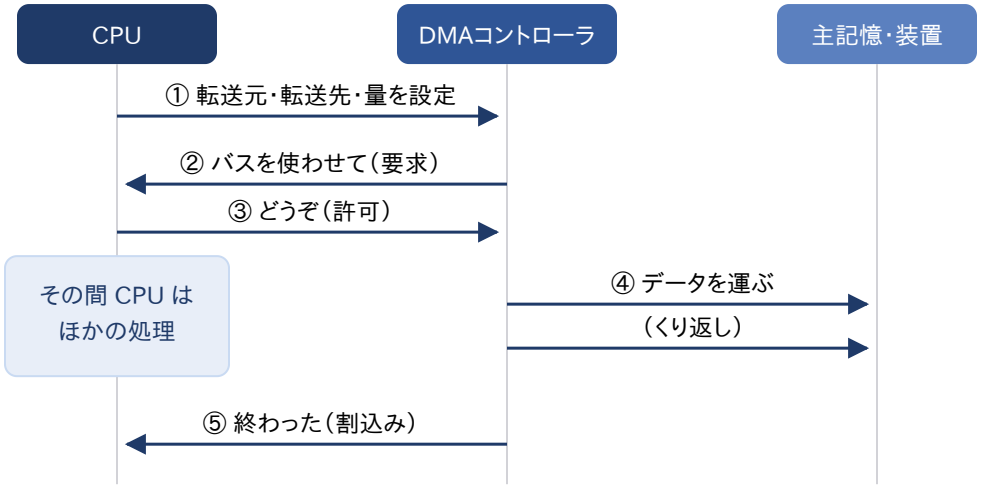
誤りの選択肢は全部、マイコンの中にある**ほかの部品の正しい説明**。部品の名前と役目を1対1で覚える。

**デバイスドライバ**：OSと装置の違いを吸収するソフト。**プラグアンドプレイ**＝つなぐと自動で設定、**ホットプラグ**＝電源を入れたまま抜き差し。



詳しく：応用情報.com 用語集 3-1 プロセッサ ／ 応用情報.com 用語集 5-1 オペレーティングシステム

# 📦 DMA転送の流れ：CPUは「頼んで、終わりの合図を待つ」だけ




| バスの使い方の違い | |
|---|---|
| **バースト**（ブロック転送） | 許可をもらったら全部まとめて運ぶ。その間 CPU はバスを使えない |
| **サイクルスチール** | 1回分運ぶたびにバスを CPU に返す。CPU の命令と交互に進む |

**DMA は4回出た：**H22秋 問12・H23秋 問13・H25秋 問12・R7春 問9。誤りの選択肢は、DSP（積和演算）、MMU（仮想メモリ）、タイマ（時間を数える）、パイプライン（命令を重ねる）、メモリマップドI/O（装置のレジスタに番地を割り当てる）。

出典：IPA 平成22年度秋期 問12・平成23年度秋期 問13・平成25年度秋期 問12・令和7年度春期 問9（午前）／Wikipedia「Direct memory access」（バースト・サイクルスチール、BR・BG の信号）

第4章

# 入出力装置

ディスプレイ ／ 入力装置とプリンタ ／ 磁気ディスクのアクセス時間 ／ RAID ／ 磁気テープ






# 📺 ディスプレイは「自分で光るか、光を通すか、反射させるか」で見分ける

| 方式 | 表示のしかた | 特徴 |
|---|---|---|
| **液晶** | バックライトの光が**通る量**を液晶で調節 | 自分では光らない。TFT（画素ごとにスイッチ）とSTN |
| **有機EL** | 電圧をかけると**自分で光る**素子 | 薄い・曲がる・黒が真っ黒 |
| プラズマ | 放電で出た紫外線で蛍光体を光らせる | 大画面。今はほぼない |
| CRT | 電子ビームを蛍光体に当てる | ブラウン管 |
| **電子ペーパー** | 着色した帯電粒子を電極に集める（電気泳動型） | 表示を保つのに電力不要 |
| DMD | 微小な鏡の向きを変えて反射 | プロジェクタ（DLP） |

R5春 問12（**有機EL**）の正解は「電圧をかけて発光素子を発光」（ア）。R4秋 問11（**電子ペーパー**）の正解は「着色した帯電粒子を集める」（イ）。どちらも、誤りの選択肢は表の残りの行。

**画面1枚のデータ量**

**横の画素数 × 縦の画素数 × 1画素のビット数 ÷ 8**

| 名前 | 画素数 | 24ビットカラーで |
|---|---|---|
| VGA | 640×480 | 約0.9Mバイト |
| SVGA | 800×600 | 約1.4Mバイト |
| XGA | 1,024×768 | 約2.4Mバイト |
| フルHD | 1,920×1,080 | 約6.2Mバイト |

ビット数のまま答える（8倍大きい）誤りに注意。**インタレース**＝1本おきに2回で描く、**ノンインタレース**＝順に全部描く。

# 📺 ディスプレイを断面で比べる。光をどこで作るかがすべて

| 液晶（LCD） |
| --- |
| 偏光フィルタ |
| カラーフィルタ（赤緑青） |
| 液晶（電圧で向きが変わる） |
| TFT（画素ごとのスイッチ） |
| 偏光フィルタ |
| バックライト ← 光はここ |

| 有機EL（OLED） |
| --- |
| 保護ガラス・フィルム |
| 有機層 ← 自分で光る |
| TFT（画素ごとのスイッチ） |
| 基板（曲げられる樹脂も） |

液晶は光の**通る量**を変えるだけ。有機ELはバックライトがなく、黒は**消灯**で真っ黒。

| 電子ペーパー |
| --- |
| 透明電極 |
| 白と黒の帯電粒子 ← 外の光を反射 |
| 電極 |

電圧で粒子を上下に集める。表示を保つのに電力が要らない。

| 3D の方式（R2秋 問11） | しくみ |
| --- | --- |
| **アクティブシャッタ** | 左右の映像を**交互**に出し、眼鏡が同期して片目ずつ閉じる |
| 偏光方式 | 左右を同時に出し、偏光の向きが違う眼鏡で分ける |
| アナグリフ | 赤と青で重ね、赤青の眼鏡で見る |
| 視差バリア | 画面の細かい格子で左右に振り分ける（裸眼） |

**H22秋 問13 の型** 音を記録するとき：録音＝A/D変換（アナログ→数値）、再生＝D/A変換。音質はサンプリング周波数と量子化ビット数の両方で決まる。
例：44.1kHz × 16ビット × 2ch ＝ 1,411,200ビット/秒 → 1分で約10.6Mバイト

出典：IPA 平成21年度秋期 問13・平成27年度秋期 問11・平成28年度春期 問11・令和2年度10月 問11・平成22年度秋期 問13（午前）／Wikipedia「Liquid-crystal display」・「OLED」・「Electronic paper」。音のデータ量はClaudeが計算した

# 入力装置は「何を読むか」、プリンタは「何単位で、どう付けるか」

| 入力装置 | 読み取るもの |
|---|---|
| OCR | 紙の**文字**を文字コードに |
| OMR | マークシートの**塗った位置** |
| イメージスキャナ | 紙を**画像**として（細かさは dpi） |
| バーコードリーダ | JANコード・QRコード |
| デジタイザ | 板の上で指した位置の**座標** |
| タッチパネル | 画面に触れた位置（静電容量方式・抵抗膜方式） |
| A/Dコンバータ | センサの電圧などを数値に（D/Aはその逆） |

**Nキーロールオーバー**：たくさんのキーを同時に押しても全部読み取れる性能。

| プリンタ | しくみ | 速さの単位 |
|---|---|---|
| シリアル | 1文字ずつ | cps（字/秒） |
| ライン | 1行ずつ | lpm（行/分） |
| ページ | 1ページずつ（レーザ） | ppm（枚/分） |
| インパクト | ピンでリボンを打つ。複写伝票に強い | — |
| レーザ | 感光ドラム+トナーを熱で定着 | — |
| インクジェット | インクの粒を吹き付ける | — |
| 熱転写 | リボンのインクを熱で溶かして写す | — |
| 3Dプリンタ | 材料を層にして積み上げる | — |

印字の単位（シリアル・ライン・ページ）と、印字のしかた（インパクトかどうか）は**別の分け方**。レーザは「ページ」かつ「ノンインパクト」。

# プリンタのしくみ：インクジェットは「熱か圧電」、レーザは6つの工程

| インクジェット | インクを押し出す力 |
|---|---|
| サーマル（バブルジェット） | ヒーターで瞬間的に加熱 → **泡**ができてインクを押し出す。キヤノンの遠藤一郎氏の発想がもと |
| ピエゾ（圧電） | 電圧をかけると形が変わる**圧電素子**でインクを押し出す |

**例題** A4（8.27×11.69インチ）を 600dpi で印刷する点の数は？
8.27×600≒4,961、11.69×600≒7,016 → 約**3,480万点**。
カラーで1点24ビットなら約104Mバイト。

dpi＝1インチ（2.54cm）に並ぶ点の数。スキャナの細かさも dpi で表す。

**レーザプリンタ（電子写真方式）**

**① 帯電**
ドラムに電気を帯びさせる

**② 露光**
レーザで文字の形に電気を消す

**③ 現像**
トナーを形どおりに付ける

**④ 転写**
トナーを紙に移す

**⑤ 定着**
熱と圧力で溶かして固める

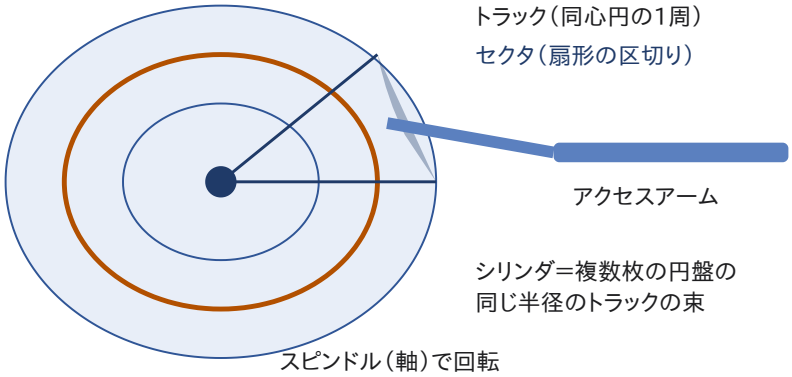
**⑥ クリーニング**
残ったトナーを落とす

1ページ分をまとめて作るので**ページプリンタ**で、ピンで打たないので**ノンインパクト**。
速さは ppm（1分あたりの枚数）。

インパクトプリンタ（ドット）だけが、カーボン複写の伝票を一度に印字できる。

出典：IPA シラバス Ver.7.2 中分類3 5.（2）出力装置の用語例／Wikipedia「Laser printing」・「Inkjet printing」（キヤノンの遠藤一郎氏のバブルジェット）。例題はClaudeが計算した

# 磁気ディスクの時間は「ヘッドを動かす＋回ってくるのを待つ＋読む」




**R3秋 問11：1,000バイトを読む平均時間**

| 時間 | 計算 | 値 |
|---|---|---|
| 平均シーク時間 | 与えられる | 10ms |
| 平均回転待ち | 1回転＝60秒÷6,000＝10ms の半分 | 5ms |
| データ転送時間 | 1,000 ÷ 10,000,000 秒 | 0.1ms |
| **平均アクセス時間** | | **15.1ms（ア）** |

誤りの選択肢は、回転待ちを**1回転分**にした 20.1、転送を**1桁大きく**した 16.0、両方を誤った 21.0。

**デフラグ**はファイルを連続した場所に並べ直してシークを減らす。SSDには不要（書換え回数を増やすだけ）。



# 平均アクセス時間を手順どおりに：位置決め＋半回転＋転送

**H24秋 問12** 毎分6,000回転、平均位置決め時間20ミリ秒、1トラック20kバイト。4kバイトを1ブロック読む平均アクセス時間は？

1. **1回転の時間**：6,000回転/分 ＝ 100回転/秒 → 1回転 10ミリ秒
2. **平均回転待ち**：半回転 ＝ 5ミリ秒（運が良ければ0、悪ければ1回転。平均は半分）
3. **転送時間**：1回転で20kバイト読める → 4kバイトは 10 × 4/20 ＝ 2ミリ秒
4. **合計**：20 ＋ 5 ＋ 2 ＝ **27ミリ秒（ウ）**

誤りの選択肢：20（位置決めだけ）、22（回転待ちを忘れた）、32（回転待ちを1回転にした）。

**回転数と平均回転待ち時間**

| 回転数 | 1回転 | 平均回転待ち |
|---|---|---|
| 5,400rpm | 11.1ミリ秒 | 5.6ミリ秒 |
| 7,200rpm | 8.3ミリ秒 | 4.2ミリ秒 |
| 10,000rpm | 6.0ミリ秒 | 3.0ミリ秒 |
| 15,000rpm | 4.0ミリ秒 | 2.0ミリ秒 |

**平均回転待ち ＝ 60 ÷ 回転数 ÷ 2**

東芝のエンタープライズ向け HDD には 10,000rpm（10Krpm）の製品群がある。SSD には回転も位置決めもないので、ランダムな読出しが桁違いに速い（記憶階層の表で HDD 4.16ミリ秒、SSD 0.2ミリ秒）。

出典：IPA 平成24年度秋期 応用情報技術者試験 午前 問12／Wikipedia「Hard disk drive performance characteristics」（平均回転待ち＝最大の半分、最大＝60/rpm）／東芝デバイス&ストレージ「データセンター・エンタープライズ向けHDD」。表の値はClaudeが式から計算した

# RAID5は「1台分をパリティに」、テープは「ブロックにまとめてすき間を減らす」

## R4春 問11：8Tバイト×6台、ホットスペア1台込みのRAID5




使えるのは 4台分 × 8Tバイト = **32Tバイト（イ）**

パリティは実際には全部のディスクに**分散して**置く（図は量を表したもの）。1台壊れても、残りから計算して元に戻せる。予備（ホットスペア）はふだんデータを入れないので容量に入れない。

40（パリティを引き忘れ）、48（予備もパリティも引き忘れ）が誤りの選択肢。

## 磁気テープのブロック化




| 用語 | 意味 |
|---|---|
| 論理レコード | 1件分のデータ（社員1人分など） |
| 物理レコード（ブロック） | まとめて1回で読み書きする単位 |
| ブロック化因数 | 1ブロックに入れる件数（図では4） |
| IBG | ブロックの間のすき間。ブロックを大きくすると数が減り、容量が有効に使える |

**MAID**：大量のディスクのうち、使っていないものの回転を止めて電力を減らすしくみ。



詳しく：応用情報.com 用語集 3-5 入出力装置 ／ 応用情報.com 用語集 5-3 ファイルシステム

# RAIDは容量と「何台まで壊れてよいか」の表で覚える。復元は XOR

| 4TB のディスク4台で組んだとき（RAID1 は2台1組） | | | |
|---|---|---|---|
| RAID | しくみ | 使える容量 | 壊れてよい台数 |
| RAID0 | ストライピング（分けて並べる） | 16TB | 0台 |
| RAID1 | ミラーリング（同じものを2台に） | 2台で4TB | 1台 |
| RAID5 | パリティを全ディスクに分散 | **12TB**（1台分減る） | **1台** |
| RAID6 | パリティを2種類 | 8TB（2台分減る） | 2台 |
| RAID1＋0 | ミラーしたものをストライピング | 8TB | 1台（組が違えば2台） |

RAID は「安価なディスクを束ねて速く・壊れにくく」という1988年のバークレー校の論文で整理された。

**例題** RAID5 で D1＝1010、D2＝0110、D3＝0011 のパリティ P と、D2 が壊れたときの復元。

| | |
|---|---|
| P ＝ D1⊕D2⊕D3 | 1010⊕0110＝1100、⊕0011＝**1111** |
| D2 ＝ D1⊕D3⊕P | 1010⊕0011＝1001、⊕1111＝**0110** |

XOR は「同じなら0、違えば1」。どの1台が欠けても、残り全部の XOR で戻る。

| 磁気テープ LTO の互換（LTO公式） | |
|---|---|
| LTO-9 ドライブ | LTO-8・9 を読み書き |
| LTO-10 ドライブ | LTO-10 だけ（30TB／40TB の媒体） |

出典：IPA 令和4年度春期 午前 問11（RAID5の容量）／Wikipedia「Standard RAID levels」（Patterson, Gibson, Katz「A Case for Redundant Arrays of Inexpensive Disks」1988）／LTO Program「LTO Generation Compatibility」。例題はClaudeが計算した

# 試験で狙われやすい比較10選（1/2）

| | 比較 | 見分ける一言 |
|---|---|---|
| 1 | ライトスルー vs ライトバック | **両方に**書く（主記憶はいつも最新）／**キャッシュだけ**に書き、追い出すとき書き戻す |
| 2 | スーパースカラ vs VLIW | 並べるのは**実行時のハードウェア**／**コンパイル時のコンパイラ** |
| 3 | 分岐予測 vs 投機実行 | 分岐先を**予想する**／予想した先を**実行まで**進める（外れたら取り消す） |
| 4 | ヘテロジニアス vs ホモジニアス | **違う**種類のコア（スマホ）／**同じ**種類のコア |
| 5 | DRAM vs SRAM | コンデンサ・**リフレッシュ要**・主記憶／フリップフロップ・不要・キャッシュ |

1・4 は、過去問で正解と誤りの選択肢として**同じ問題に並んだ**組合せ（R3春 問12、R5秋 問8）。2・3 は、過去問の正解（R4春 問8 の VLIW、R5秋 問9 の投機実行）と取り違えやすい相手。

# 試験で狙われやすい比較10選（2/2）

| | 比較 | 見分ける一言 |
|---|---|---|
| 6 | ダイレクトマップ vs フルアソシエイティブ | 置き場所が**1か所**に決まる／**どこでも**置ける（セットアソシエイティブはその中間） |
| 7 | メモリインタリーブ vs ディスクキャッシュ | 主記憶を**バンク**に分けて並列に／主記憶と**磁気ディスク**の間のバッファ |
| 8 | ウェアレベリング vs ライトアンプリフィケーション | 書込みを散らして寿命を延ばす**技術**／書込み量が増えて寿命が縮む**現象** |
| 9 | 内部割込み vs 外部割込み | 原因が**プログラムの中**（0除算・SVC）／**外**（入出力完了・タイマ・電源異常） |
| 10 | プラグアンドプレイ vs ホットプラグ | つなぐと**自動で設定**／**電源を入れたまま**抜き差し |

**おまけ：**有機EL（自分で光る）・液晶（光を通す量）・電子ペーパー（粒を集める）、即値（データそのもの）・直接（番地）・間接（番地の中身が番地）、DMA（転送）・DSP（積和）・MMU（メモリ管理）。

# 出典（1/2）：試験・規格団体・メーカー

**IPA（試験の公式資料）**

応用情報技術者試験 午前 問題冊子・解答例 平成21年度春期～令和7年度秋期の33回分（IPA 過去問題。許諾・使用料なしで利用できる）。この分野の125問を読み、分野は応用情報技術者試験.comの分類で確かめた／応用情報技術者試験（レベル3）シラバス Ver.7.2 中分類3

**規格団体**

USB-IF（USB4・USB 3.2・USB Power Delivery）／SD Association（容量・バス速度）／NVM Express／VESA（DisplayPort）／HDMI Licensing（HDMI 2.2）／NFC Forum／Bluetooth SIG／CSA（Zigbee）／Wi-Fi Alliance（Wi-Fi 7）／LTO Program／RISC-V International

**メーカー・研究機関**

Arm（big.LITTLE）／インテル（ハイパースレッディング）／Apple（M1・ユニファイドメモリ）／Google Cloud（TPU）／IBM（量子コンピュータ）／D-Wave（量子アニーリング）／キオクシア（多値NAND）／東芝デバイス＆ストレージ（HDD）／TOP500（2025年11月）

**論文**

Arute et al.「Quantum supremacy using a programmable superconducting processor」Nature 574, 2019

すべて2026年11月1日に開いて内容を確かめた。数値は各ページに書かれていたもの

# 出典（2/2）：解説・辞典とこの資料の作り方

### Wikipedia（英語版。数値と定義の確認に使った項目）

Memory hierarchy（実測の待ち時間）／CPU cache／Cache replacement policies／Cache coherence／Memory interleaving／Hamming code／ECC memory／Parity bit／Dynamic random-access memory／DDR4 SDRAM／DDR5 SDRAM／Flash memory／Wear leveling／Trim／Phase-change memory／Ferroelectric RAM／SD card／Memory management unit／Virtual memory／Instruction pipelining／Hazard／Branch predictor／Out-of-order execution／Speculative execution／Superscalar processor／Very long instruction word／Simultaneous multithreading／Flynn's taxonomy／SIMD／Vector processor／Amdahl's law／Symmetric multiprocessing／Non-uniform memory access／Processor power dissipation／Power gating／Dynamic frequency scaling／Reduced instruction set computer／Microcode／Adder／Two's complement／Cycles per instruction／Instructions per second／FLOPS／Endianness／Tensor Processing Unit／Quantum annealing／PCI Express／NVM Express／SATA／USB／Thunderbolt／DisplayPort／HDMI／LPWAN／Direct memory access／Liquid-crystal display／OLED／Electronic paper／Laser printing／Inkjet printing／Hard disk drive performance characteristics／Standard RAID levels／Linear Tape-Open

### 参照リンク

各ページのリンク先は、IT用語辞典 e-Words、応用情報技術者試験.com（用語集・過去問解説）。2026年11月1日にリンク先が開けることを確かめた

### イラスト

Microsoft Fluent Emoji（MIT License。全文は tools/slides/img/comp/LICENSE-fluentui-emoji.txt）／いらすとや（https://www.irasutoya.com/）

### 作成

上の資料を読んでClaudeがまとめた。解説・例題・図はClaudeが自分の言葉と計算で作ったもので、IPA の公式解説ではない。ほかのサイトの解説文は写していない。サイトの用語集（349語）と問題演習（87問）も同じ資料から作っている

### 読めなかった資料

JEDEC（DDR の規格本文）と PCI-SIG（PCIe の規格本文）はサイトに接続を断られたため、数値は Wikipedia の該当項目（それぞれ規格を出典に挙げている）で確かめた。RAID の原論文（バークレー校の技術報告 PDF）は画像だけで読めなかった